GRAVELLARD LEA

11904860

Année scolaire 2025 - 2026

SORBONNE DATA ANALYTICS

Promotion 2025-2026

MEMOIRE DE FIN D'ETUDES

Modélisation prédictive de la qualité de l'air : dans quelle mesure le Machine Learning permet-il de prédire la qualité de l'air à partir des seules données météorologiques, indépendamment des mesures directes de polluants ?

Introduction **4**
1. État de l'Art et Contexte Scientifique **8**
1.1. Les approches déterministes (Modèles physiques) 8
1.2. L'avènement du Machine Learning (Approches statistiques) 9
1.2.1. Une rupture : du déterminisme à l'approche guidée par la donnée 10
L'avènement de l'Open Data et la richesse informationnelle 10
Performance computationnelle et réactivité en temps réel 10
La capture des phénomènes non-linéaires et des effets de seuil 11
1.2.2. Panorama des architectures algorithmiques dominantes 11
Les modèles de régression classiques (Linéaires et Polynomiales) 11
Les modèles d'ensemble : La puissance de la décision collective 11
Les Réseaux de Neurones et le Deep Learning 12
1.2.3. Les solutions du marché (Benchmarks) 13
Google Air Quality API : La fusion de données à grande échelle 13
IQAir (AirVisual) : L'agrégation et la validation par l'IA 13
OpenWeatherMap : L'accessibilité par API 13
2. Théorie Algorithmique **15**
2.1. La Régression Logistique 16
2.1.1. Fonctionnement Mathématique 17
L'Équation Linéaire 18
La Fonction Sigmoïde (La Probabilité) 18
Le Seuil de Décision et l'extension Multinomiale 19
2.1.2. Fonctionnement informatique 19
2.2. Le Random Forest 20
2.2.1. Fonctionnement Mathématique 20
L'Arbre de Décision 20
Le principe de la Forêt (Ensemble Learning) 21
2.2.2. Fonctionnement informatique 23
2.3. Le XGBoost 24
2.3.1. Fonctionnement Mathématique 24
L'Apprentissage Résiduel (Correction d'erreurs) 24
La Descente de Gradient 24
La Régularisation (Le secret de XGBoost) 24
2.3.2. Fonctionnement informatique 25
3. Application et Développement — Le cœur technique **27**
3.1. Le choix des technologies et langages 27
3.1.1. L'IDE 27
3.1.2. Les langages 27
Python : Le moteur de l'analyse et de l'intelligence 27
HTML : L'interface et la restitution 28
3.2. Les librairies 28
3.2.1. L'importation des librairies 28
3.2.2. L'environnement virtuel 29

3.3. Le jeu de données....29
3.3.1. Contexte....29
3.3.2. Exploration....30
3.3.3. Visualisations....33
3.4. Pré-traitement des données....35
3.4.1. Les Opérations et choix sur les données....35
Suppression de variables....35
Formatage des variables....36
Modification d'une variable....37
Renommage des variables....38
Création de variables....38
3.4.2. Visualisations....38
Distribution de l'indice de qualité de l'air US EPA par continent....39
Boxplots des variables numériques....40
Distribution de l'indice de qualité de l'air US EPA par année....41
3.5. Modélisation....41
3.5.1. Split....42
3.5.2. Modèles....43
Logistic Regression....43
Random forest....44
XGBoost....44
Analyse et choix du modèle final....44
3.5.3. Cross-validation....45
3.5.3. Overfitting....46
Choix final du modèle....48
3.5.4. Interprétabilité....49
Analyse de l'importance des variables (Feature Importance)....50
3.5.5. Conclusion....50
Performance et fiabilité du modèle....50
Les leviers météorologiques de la pollution....51
Réponse à la problématique....51
4. Solution Opérationnelle : De l'Algorithme à l'Application....52
4.1. Architecture Logicielle et Environnement....52
4.2. Interface Utilisateur : Les quatre piliers de l'outil....52
4.3. Accessibilité et Reproductibilité....52
5. Conclusion et Perspectives....53
6. Bibliographie et Webographie....55
6.1. Institutions, Rapports et Données....55
6.2. Littérature Scientifique et Théorie du Machine Learning....55
6.3. Documentation Technique et Bibliothèques Python....55
5.4. Concepts Généraux et Webographie....56

Introduction

Nous vivons depuis plusieurs siècles, une urbanisation que l'on peut qualifier d'exponentielle. L'urbanisation est un mouvement historique de transformation de la société que l'on peut définir comme l'augmentation du nombre d'habitants en ville par rapport à l'ensemble de la population. Cette urbanisation est multifactorielle. Le facteur premier fut l'industrialisation, soit, le passage d'une économie à prépondérance agricole à une économie à prépondérance industrielle.

La concentration de la population s'est alors totalement inversée, passant d'une population majoritairement rurale à une population majoritairement urbaine. Le pourcentage de la population mondiale habitant en ville passant de 3,4 % en 1800 à 55% en 2018. Le point de bascule de 50% fut atteint en 2007 ! Selon l'étude des Nations Unies *'World urbanization prospects 2025'* : D'ici 2050, environ 70 % de la population mondiale vivra dans des zones urbaines (contre environ 57 % aujourd'hui) et l'augmentation de la population urbaine sera de près de 2,3 milliards de personnes.

Cette transformation de la société s'accompagne de nombreux enjeux socio-économiques, culturels, environnementaux et politiques.

Parmi ces défis, l'enjeu environnemental s'impose comme l'un des plus critiques. Cette densification urbaine sans précédent s'accompagne d'une intensification des activités humaines : transports routiers, chauffages domestiques, activités industrielles, etc. Cette concentration d'activités génère un cocktail complexe de polluants atmosphériques, transformant les centres urbains en foyers d'émissions massives. Selon l'Organisation Mondiale de la Santé (OMS), la pollution de l'air ambiant est désormais responsable de plus de 7 millions de décès prématurés chaque année, s'affirmant comme un des risques environnementaux majeur pour la santé humaine.

Le risque environnemental ne se limite pas à la simple émission de polluants, il réside dans la capacité de l'atmosphère à disperser ou, au contraire, à piéger ces substances. La qualité de l'air que nous respirons est le résultat d'une interaction entre les sources de pollution et les conditions météorologiques.

Les variables météorologiques agissent comme de véritables régulateurs de l'air urbain :

- **Le vent :** Facteur de dispersion, sa vitesse et sa direction déterminent si les polluants sont dilués ou s'ils s'accumulent au-dessus des habitations.
- **Le rayonnement solaire et les UV :** Ils jouent le rôle de catalyseurs[1], déclenchant des réactions photochimiques[2] qui transforment des gaz précurseurs en polluants secondaires plus nocifs.

---

[1] **Catalyseur** : Un catalyseur est une substance qui provoque ou accélère une réaction chimique sans être elle-même modifiée ou consommée par cette réaction.

[2] Une **réaction photochimique** est une transformation chimique déclenchée par l'absorption d'énergie lumineuse, principalement le rayonnement ultraviolet solaire. En milieu urbain, ce processus est le moteur qui casse les molécules de gaz d'échappement pour créer des polluants secondaires nocifs comme l'ozone.

- **La température et l'humidité :** Ces facteurs régissent la stabilité de l'air. Le phénomène d'inversion thermique[3], par exemple, crée un 'couvercle' d'air chaud qui emprisonne les polluants au niveau du sol, provoquant des pics de pollution dangereux pour les populations vulnérables.

Pour combattre ce fléau environnemental, il faut dans un premier temps le mesurer. Pour mesurer la pollution de l'air, il existe un indicateur qui se nomme AQI (Air Quality Index), développé par l'Agence américaine de protection de l'environnement (Environmental Protection Agency). Il s'agit d'une mesure des concentrations de polluants atmosphériques dans la pollution de l'air ambiant et leurs risques de santé associés. Les polluants mesurés dans la formule sont au nombre de cinq principaux, chacun ayant des propriétés et des impacts distincts sur l'organisme et l'environnement :

- **Les Particules fines (PM2.5 et PM10)** : Ce sont des micro-particules. Les PM2.5 (diamètre inférieur à 2,5 micromètres) sont les plus dangereuses, car leur finesse leur permet de pénétrer profondément dans les alvéoles pulmonaires et de passer dans le système sanguin.
- **L'Ozone de basse altitude (O3) :** À l'inverse de la 'couche d'ozone' protectrice en haute altitude, l'ozone au sol est un polluant secondaire agressif, formé par la réaction chimique entre les oxydes d'azote et les rayons UV du soleil.
- **Le Dioxyde d'Azote (NO2) :** Principalement émis par la combustion des moteurs thermiques (trafic routier), c'est un gaz irritant majeur pour les voies respiratoires.
- **Le Monoxyde de Carbone (CO) :** Gaz inodore et incolore issu de combustions incomplètes, il réduit la capacité du sang à transporter l'oxygène.
- **Le Dioxyde de Soufre (SO2) :** Produit par la combustion de combustibles fossiles contenant du soufre (industrie, transport maritime), il est responsable des pluies acides et d'inflammations pulmonaires.

L'AQI fonctionne comme une échelle graduée de 0 à 500. Plus la valeur est élevée, plus la pollution est importante et les risques pour la santé sérieux. L'agence américaine a défini six catégories de couleurs pour faciliter la lecture rapide par le grand public :

1. **Bon (0-50) :** Vert. La qualité de l'air est satisfaisante.
2. **Modéré (51-100) :** Jaune. Qualité acceptable ; toutefois, les personnes très sensibles peuvent ressentir des symptômes légers.
3. **Mauvais pour les groupes sensibles (101-150) :** Orange. Les personnes souffrant de maladies respiratoires ou cardiaques sont à risque.

---

[3] **Inversion thermique** : Il s'agit d'un phénomène météorologique où la structure normale de l'atmosphère est inversée. En temps ordinaire, la température de l'air diminue à mesure que l'on monte en altitude. Lors d'une inversion, une couche d'air chaud se superpose à une couche d'air froid stagnant au niveau du sol.

Cette configuration agit comme un « couvercle » invisible : l'air froid, plus dense et plus lourd, ne peut pas s'élever à travers la couche chaude. Par conséquent, les polluants émis au sol (gaz d'échappement, fumées industrielles, particules fines) ne sont plus évacués vers la haute atmosphère et restent prisonniers dans la zone de respiration des populations, provoquant une dégradation rapide de la qualité de l'air.

4. **Malsain (151-200) :** Rouge. Tout le monde peut commencer à ressentir des effets sur la santé.
5. **Très malsain (201-300) :** Violet. Alerte de santé publique, les risques augmentent pour l'ensemble de la population.
6. **Dangereux (301+) :** Marron. Conditions d'urgence sanitaire affectant toute la population de manière critique.

Il est crucial de noter que si l'AQI de l'EPA (utilisée dans ce mémoire) est le standard international le plus reconnu, il n'est pas unique. La mesure de la qualité de l'air n'est pas universellement harmonisée :

- **L'AQI Américain (US EPA) :** Il est souvent considéré comme le plus strict, notamment et sert de référence pour de nombreuses plateformes mondiales comme IQAir.
- **L'Indice Européen (EA QI) :** Développé par l'Agence Européenne pour l'Environnement, il utilise des paliers différents et se concentre davantage sur les concentrations horaires de cinq polluants clés.
- **Les variantes nationales :** Des pays comme la Chine ou l'Inde ont développé leurs propres indices (AQI-CN, NAAQS) pour s'adapter à leurs contextes de pollution industrielle spécifiques, avec des échelles de dangerosité souvent décalées par rapport aux normes occidentales.

Pour ce projet, le choix de l'AQI américain se justifie par sa robustesse statistique et sa large adoption dans les jeux de données mondiaux, permettant une comparaison cohérente entre les différentes métropoles étudiées.

Pour calculer cette AQI, il nous faut alors calculer les polluants qui l'a compose. Pour ce faire, ce sont des appareils de surveillance qui sont utilisés. Ces appareils de surveillance de la qualité de l'air sont équipés de capteurs conçus pour détecter des polluants spécifiques. Certains utilisent des lasers pour mesurer la densité des particules dans un mètre cube d'air, tandis que d'autres s'appuient sur l'imagerie satellitaire pour mesurer l'énergie réfléchie ou émise par la Terre. Mais un problème majeur se présente, ces appareils représentent un coût non négligeable et ne sont alors pas déployés dans de nombreux endroits, principalement dans les zones dites 'sous-développées'. En effet, tandis que les métropoles des pays développés disposent d'un maillage dense de capteurs, de vastes régions en Afrique, en Asie du Sud-Est et en Amérique latine (là où l'urbanisation est pourtant la plus rapide et la plus désordonnée) se retrouvent dans un véritable 'blackout' de données. Sans mesures, il est impossible pour les autorités locales de mettre en place des politiques de santé publique efficaces ou d'alerter les populations lors de pics de pollution critiques.

Si les capteurs chimiques sont rares et coûteux, les données météorologiques, elles, sont devenues universelles. Grâce aux réseaux de stations météo standards, aux radars et surtout à l'imagerie satellitaire, des variables comme la température, la vitesse du vent ou le taux d'humidité sont accessibles en temps réel pour quasiment chaque point du globe, et ce, à un coût dérisoire.

L'idée centrale de ce travail est alors de transformer ce flux de données météorologiques en un 'capteur virtuel'. En utilisant la puissance du machine learning, nous cherchons à savoir si la météo peut devenir le miroir de la pollution, nous permettant de déduire la qualité de l'air là où les capteurs physiques font défaut.

Nous traiterons alors dans ce mémoire de la problématique suivante : **Modélisation prédictive de la qualité de l'air : dans quelle mesure le Machine Learning permet-il de prédire la qualité de l'air à partir des seules données météorologiques, indépendamment des mesures directes de polluants ?'**

Pour répondre à cette problématique, ce travail est structuré en cinq parties majeures :

- **État de l'Art et Contexte Scientifique :** Nous explorerons d'abord le contexte de la pollution atmosphérique en comparant les approches physiques traditionnelles aux méthodes statistiques modernes du Machine Learning. Cette section inclut également un benchmark des solutions leaders du marché afin de situer notre projet dans l'écosystème actuel.
- **Théorie Algorithmique :** Nous détaillerons les fondements mathématiques et le fonctionnement informatique des trois modèles sélectionnés pour cette étude : la Régression Logistique, le Random Forest et le XGBoost.
- **Application et Développement — Le cœur technique :** Ce chapitre constitue la phase de réalisation concrète. Il décrit l'environnement de développement, le cycle de vie de la donnée et l'étape de modélisation incluant l'évaluation des performances et l'analyse de l'interprétabilité des variables.
- **Solution Opérationnelle :** Nous présenterons la concrétisation de l'étude à travers le développement d'une application interactive. Cette partie détaille l'architecture logicielle permettant de transformer un modèle abstrait en un outil fonctionnel capable de réaliser des prédictions en temps réel via API.
- **Conclusion et Perspectives :** Enfin, nous synthétisons les résultats obtenus pour répondre définitivement à la problématique, tout en ouvrant sur les limites et les évolutions possibles de notre modèle

L'intégralité du code source ayant servi à cette étude est accessible publiquement sur le dépôt GitHub suivant : https://github.com/leagravellard/Memoire.

Ce répertoire comprend non seulement le Notebook Jupyter (`notebook.ipynb`) détaillant l'exploration et l'entraînement des modèles, mais aussi le code de l'application web (`app.py`) développée avec `'Streamlit'`. Bien que ce mémoire présente les étapes déterminantes de notre démarche, il ne se veut pas exhaustif. Pour une consultation approfondie de l'intégralité des tests, des fonctions de prétraitement, des étapes intermédiaires, nous invitons le lecteur à se référer au répertoire complet cité ci-dessus.

1. État de l'Art et Contexte Scientifique

De nos jours, il existe différentes manières de prédire la pollution. Chacune ayant ses avantages et ses défauts.
Nous allons exposer dans cette partie les deux méthodes principales qui permettent la prédiction. Nous exposerons alors leurs caractéristiques et le choix pour lequel nous avons opté.

1.1. Les approches déterministes (Modèles physiques)

L'approche déterministe repose sur une compréhension mathématique et physique profonde des phénomènes atmosphériques. Ces modèles s'appuient sur des équations de mécanique des fluides (telles que les équations de Navier-Stokes[4]) et de chimie atmosphérique pour simuler le transport, la dispersion et la transformation des polluants dans l'air.

Comment ça fonctionne ?

Ces modèles, souvent appelés Modèles de Transport de Chimie (CTM - *Chemical Transport Models*), nécessitent des données d'entrée très détaillées :

- **Un inventaire des émissions :** Il faut connaître précisément la position, la hauteur et le débit de chaque source de pollution (cheminées d'usines, pots d'échappement par segment de rue, chauffage urbain, etc.).
- **Des données météorologiques tridimensionnelles[5] :** Le modèle doit intégrer la topographie (relief, bâtiments) pour calculer comment l'air rebondit ou stagne dans les 'canyons urbains'[6].

Le principal point fort de cette approche est sa capacité d'explication. Puisqu'elle repose sur les lois de la physique, elle permet de réaliser des scénarios précis (par exemple : 'Quel sera l'impact sur la qualité de l'air si l'on ferme cette autoroute ?').

Cependant, malgré leur précision théorique, les modèles physiques se heurtent à plusieurs obstacles majeurs :

---

[4] En mécanique des fluides, les **équations de Navier-Stokes** sont des équations aux dérivées partielles non linéaires qui décrivent le mouvement des fluides newtoniens (donc des gaz et de la majeure partie des liquides).
[5] L'aspect **tridimensionnel** est ici fondamental : contrairement à une simple analyse de surface, ces modèles découpent l'atmosphère en une grille de volumes (cubes) superposés. Cela permet de simuler non seulement le déplacement horizontal des polluants par le vent, mais aussi leur diffusion verticale en tenant compte du relief et de la hauteur des bâtiments (canyons urbains), facteurs déterminants dans la stagnation ou la dispersion des particules.
[6] L'expression **« canyon urbain »** est une métaphore utilisée en urbanisme et en météorologie pour décrire une rue bordée d'immeubles alignés de chaque côté, créant une forme qui ressemble à un canyon naturel creusé dans la roche.

1. **Coût de calcul :** Les calculs sont si lourds qu'ils nécessitent souvent des supercalculateurs[7]. Il est donc difficile de les faire tourner en temps réel sur une application légère.
2. **Dépendance aux données d'inventaire :** Si les données sur les sources d'émissions (trafic, industrie) sont incomplètes ou obsolètes, le modèle devient erroné, même si les équations physiques sont correctes.
3. **Mise à jour difficile :** Adapter un modèle physique à une nouvelle ville demande un travail de configuration et de paramétrage qui peut prendre des mois.

Ces modèles physiques sont aujourd'hui déployés par des organismes institutionnels de référence pour surveiller l'atmosphère à différentes échelles.

À une échelle vaste, le programme **Copernicus** de l'Union Européenne s'est imposé comme l'un des leaders mondiaux en matière de surveillance atmosphérique. Son service dédié, le **CAMS** (*Copernicus Atmosphere Monitoring Service*), fournit des données globales couvrant l'ensemble du globe. Sa force réside dans sa capacité à fusionner des observations satellitaires mondiales avec des modèles de transport chimique pour simuler, par exemple, le déplacement des fumées de méga-feux ou des nuages de poussière du Sahara d'un continent à l'autre.

En France, cette expertise se décline à travers deux acteurs majeurs :

- **L'Ineris** a développé le modèle **CHIMERE** pour répondre aux besoins de prévisions à méso-échelle[8]. Ce modèle découpe l'atmosphère en une grille tridimensionnelle et résout pour chaque maille des systèmes d'équations différentielles complexes (dynamique des fluides, chimie gazeuse). Cette précision théorique exige toutefois des capacités de calcul considérables et des inventaires d'émissions constamment mis à jour.
- **Airparif** assure la surveillance de la qualité de l'air en Île-de-France. L'organisme s'appuie sur ces modèles déterministes pour croiser les émissions locales et la météo afin de produire des cartographies régionales quotidiennes.

Ces exemples illustrent parfaitement la puissance explicative de l'approche déterministe, mais ils soulignent également la lourdeur des infrastructures et des ressources spécialisées qu'elle requiert. C'est précisément face à cette rigidité et ce besoin massif de puissance de calcul que le monde de la donnée s'est tourné vers une alternative plus agile : l'approche statistique basée sur le Machine Learning.

1.2. L'avènement du Machine Learning (Approches statistiques)

Contrairement aux approches déterministes qui tentent de reproduire les lois de la physique, l'approche statistique, et plus particulièrement le Machine Learning, repose sur l'apprentissage par l'exemple.

---

[7] Un **supercalculateur** est un ordinateur conçu pour atteindre les puissances de calcul les plus élevées possibles avec les technologies du moment.

[8] En météorologie, la **méso-échelle** décrit les phénomènes qui se passent à une échelle plus petite que l'échelle synoptique, celui de la circulation atmosphérique générale, mais plus grand que celui des nuages individuels traités par la micro-échelle. Pratiquement, on parle alors de dimensions horizontales entre 2 km et 2 000 km ce qui veut dire qu'on y traite de phénomènes tels les lignes de grain, les complexes convectifs de méso-échelle (CCM) et autres orages organisés.

Le Machine Learning, qu’est-ce que c’est ?

Le Machine Learning est une branche de l’intelligence artificielle qui repose sur des méthodes mathématiques et statistiques pour permettre à un programme informatique d’améliorer ses performances sur une tâche donnée, non pas en étant explicitement programmé pour chaque cas, mais en apprenant à partir de données. Autrement dit, plutôt que de coder des règles à la main, on fournit au modèle un grand volume d’exemples passés, et c’est lui qui en extrait les régularités et les patterns sous-jacents.

Cette capacité à détecter automatiquement des structures complexes dans les données ouvre des perspectives particulièrement intéressantes pour la prédiction de la qualité de l’air, comme en témoigne le changement de paradigme qui s’est opéré dans le domaine.

1.2.1. Une rupture : du déterminisme à l'approche guidée par la donnée

Le passage des modèles physiques aux méthodes statistiques marque une transition fondamentale dans les sciences environnementales. Là où le physicien cherche à modéliser la cause (l'émission d'une particule), le data scientist se concentre sur l'observation de l'effet (la mesure finale de l'AQI). Cette évolution repose sur trois piliers majeurs qui justifient aujourd'hui la puissance opérationnelle du Machine Learning pour le suivi environnemental des zones urbaines.

L’avènement de l’Open Data et la richesse informationnelle

L'essor de l'apprentissage automatique est intrinsèquement lié à la démocratisation des données massives.

Auparavant, la modélisation était freinée par la difficulté d'obtenir des inventaires d'émissions industrielles précis, souvent protégés ou inexistants. Aujourd'hui, des programmes mondiaux comme Copernicus (Union Européenne) ou la NASA fournissent des flux de données météorologiques et satellitaires en accès libre. Cette abondance permet d'entraîner des algorithmes sur des contextes géographiques extrêmement variés, rendant les modèles plus généralisables et moins dépendants d'infrastructures locales coûteuses.

Performance computationnelle et réactivité en temps réel

Un avantage essentiel du Machine Learning réside dans la dissociation entre le temps d'entraînement et le temps de prédiction. Si l'apprentissage du modèle (le *training*) peut être gourmand en ressources, la phase d'inférence (le calcul d'une prédiction à partir de nouvelles données) est quasi instantanée. Cette légèreté algorithmique est ce qui permet de passer d'un calcul sur supercalculateur à une intégration fluide dans des outils légers comme des dashboards ou des applications mobiles, répondant ainsi au besoin de réactivité immédiate des citoyens face aux pics de pollution.

La capture des phénomènes non-linéaires[9] et des effets de seuil

La pollution atmosphérique est un système dynamique complexe, souvent qualifié de 'chaotique'. Les relations entre les variables ne sont pas proportionnelles : par exemple, une augmentation de 2°C de la température peut n'avoir aucun impact jusqu'à atteindre un point de rupture photochimique déclenchant un pic d'ozone. Les modèles statistiques classiques (régression linéaire) échouent à capturer ces cassures. À l'inverse, les algorithmes de Machine Learning, par leur structure (notamment les arbres de décision), excellent dans l'identification de ces interactions non-linéaires et de ces seuils critiques, offrant ainsi une précision bien supérieure lors d'épisodes météorologiques extrêmes.

Cette rupture de paradigme, privilégiant l'observation de l'effet sur la modélisation de la cause, déplace le défi technique du champ de la physique pure vers celui de l'architecture logicielle. Dès lors que l'on accepte que la donnée météorologique contient l'empreinte de la pollution, la question n'est plus de savoir si l'on peut prédire, mais quel algorithme possède la structure mathématique la plus adaptée pour traiter ces flux d'informations. C'est cette nécessité de sélectionner l'outil le plus performant et le plus interprétable qui nous amène à analyser les différentes familles de modèles cohabitant dans l'état de l'art actuel.

1.2.2. Panorama des architectures algorithmiques dominantes

Dans l'état de l'art actuel de la science des données environnementales, trois grandes familles de modèles cohabitent, chacune répondant à des contraintes spécifiques.

Les modèles de régression classiques (Linéaires et Polynomiales)

Ces modèles constituent la 'ligne de base' (*baseline*) historique. Bien que très économes en puissance de calcul et parfaitement transparents, ils reposent sur l'hypothèse que la variation d'une variable météorologique entraîne une réponse proportionnelle de la pollution. Or, l'atmosphère ne réagit pas de manière linéaire. Ces modèles sont donc souvent relégués à des études préliminaires ou à des contextes où seules deux ou trois variables interagissent, mais ils s'avèrent insuffisants pour capturer la complexité d'un environnement urbain multidimensionnel.

Les modèles d'ensemble : La puissance de la décision collective

Cette catégorie regroupe les algorithmes les plus performants pour la classification de données tabulaires. Le principe fondamental de l'Ensemble Learning est de ne pas s'appuyer sur un seul modèle de prédiction (un seul 'expert'), mais de combiner les résultats d'une multitude d'arbres de décision. Cette approche permet de compenser les erreurs individuelles de chaque arbre pour obtenir un verdict final beaucoup plus stable et précis.

On distingue deux stratégies majeures au sein de cette famille :

- **Le Bagging (cas du Random Forest) :** Ici, les arbres de décision travaillent en parallèle et de manière indépendante. En moyennant les prédictions de centaines d'arbres, cet algorithme réduit drastiquement le risque de sur-apprentissage

[9] Un phénomène est dit **non-linéaire** lorsque la réponse (le résultat) n'est pas directement proportionnelle à la cause (la variation d'une donnée).

(*overfitting*). Sa force réside dans sa robustesse face aux données 'bruitées' (erreurs de mesure) ou aux capteurs défaillants.

- **Le Boosting (cas du XGBoost) :** Contrairement au Random Forest, les arbres sont construits en séquence. Chaque nouvel arbre est entraîné pour corriger spécifiquement les erreurs commises par le précédent. C'est une méthode extrêmement précise, souvent utilisée pour maximiser les performances de prédiction.

L'atout de l'interprétabilité : L'avantage majeur de ces architectures d'ensemble par rapport aux réseaux de neurones est leur capacité à calculer la « Feature Importance ». Cette fonctionnalité offre une vue claire sur les variables physiques (UV, Vent, Humidité) qui influencent réellement la prédiction de l'AQI, rendant le modèle explicable pour les utilisateurs finaux.

Les Réseaux de Neurones et le Deep Learning

Inspiré par l'architecture biologique du cerveau humain, le Deep Learning repose sur l'empilement de multiples couches de « neurones » artificiels. Chaque couche reçoit des informations de la précédente, les transforme via des calculs mathématiques complexes (poids et biais), et transmet le résultat à la couche suivante.

À la différence des arbres de décision qui suivent des règles logiques simples, les réseaux de neurones apprennent des représentations abstraites des données. Plus on ajoute de couches (d'où le terme 'profond'), plus le modèle est capable de saisir des nuances infimes dans les données de très gros volumes.

Ces modèles sont capables de performances prédictives inégalées sur de très larges volumes de données. Cependant, ils présentent deux inconvénients majeurs pour un projet d'ingénierie agile :

- **L'effet 'Boîte Noire' :** Comme le modèle réalise des millions de micro-calculs entre ses couches, il est extrêmement complexe, voire impossible, d'expliquer précisément quelle variable météo a fait basculer la prédiction vers un seuil 'Dangereux'. Pour une application destinée au public, ce manque de transparence est un point faible majeur.
- **Le coût de maintenance :** Ils nécessitent des phases d'entraînement longues et des infrastructures (GPU[10]) plus lourdes, ce qui peut freiner un déploiement sur des serveurs légers ou des solutions Cloud d'entrée de gamme.

—

[10] **GPU (Graphics Processing Unit) :** Processeur spécialisé capable de traiter des milliers de calculs en parallèle. Indispensable pour le Deep Learning, son coût de location ou d'achat est nettement supérieur à celui d'un processeur classique (CPU).

Critère	Régression Linéaire	Random Forest	Deep Learning
Précision	Faible (Linéaire)	Forte (Non-linéaire)	Très Forte
Interprétabilité	Totale	Bonne (Feature Importance)	Très Faible (Boîte Noire)
Rapidité d'entraînement	Instantanée	Modérée	Longue
Résistance aux erreurs	Faible	Excellente	Moyenne

La compréhension des différents modèles de Machine Learning permet de saisir la complexité technique du sujet. Cependant, l'efficacité de ces algorithmes ne se limite pas à la recherche académique. Aujourd'hui, des acteurs majeurs de la technologie et des services environnementaux ont déjà franchi le pas de l'industrialisation. Analyser ces solutions de marché est essentiel pour situer notre projet dans l'écosystème actuel et comprendre comment les leaders du domaine parviennent à fournir des prédictions à l'échelle mondiale.

1.2.3. Les solutions du marché (Benchmarks)

L'efficacité des modèles de Machine Learning a permis l'émergence d'acteurs technologiques majeurs qui transforment ces concepts mathématiques en services accessibles à l'échelle mondiale.

Google Air Quality API : La fusion de données à grande échelle

Google a récemment intégré la qualité de l'air comme une couche d'information stratégique dans son application 'Maps'. Leur approche repose sur un modèle de Machine Learning complexe qui fusionne des sources hétérogènes : stations de surveillance gouvernementales, données de trafic routier en temps réel, imagerie satellitaire et modèles de dispersion atmosphérique.

- **Leur force :** Google utilise l'IA pour effectuer du « Nowcasting » (prédiction immédiate), capable de 'boucher les trous' statistiques entre deux stations de mesure distantes de plusieurs kilomètres, offrant ainsi une continuité d'information là où il n'y a aucun capteur.

IQAir (AirVisual) : L'agrégation et la validation par l'IA

IQAir s'est imposé comme la référence mondiale en matière de transparence des données. Leur plateforme agrège des données provenant aussi bien de stations de référence étatiques que de micro-capteurs citoyens 'low-cost'.

- **Leur spécificité :** Comme ces capteurs bon marché sont souvent soumis à des erreurs de mesure (dues à l'humidité ou à la poussière), IQAir utilise des algorithmes d'apprentissage automatique pour calibrer et valider les données en temps réel avant leur publication. Ils transforment ainsi une donnée 'bruitée' en une information sanitaire fiable.

OpenWeatherMap : L'accessibilité par API

OpenWeatherMap, la solution retenue pour l'alimentation en temps réel de notre application, propose une 'Air Pollution API' globale. Leur modèle s'appuie sur le système de surveillance atmosphérique européen (CAMS) et des modèles de chimie de transport mondiaux.

- **L'intérêt pour notre projet :** Cette solution illustre parfaitement la démocratisation de l'information environnementale. En exposant ces données via une interface de programmation (API), ils permettent à des développeurs et ingénieurs de construire des outils d'aide à la décision.

L'étude des solutions actuelles montre que l'intelligence artificielle est l'outil le plus efficace pour prédire la pollution là où les capteurs manquent. Cependant, tous les algorithmes ne se valent pas. Pour trouver la solution la plus performante pour notre projet, il est nécessaire de comparer plusieurs approches.

C'est pourquoi nous avons choisi de tester trois modèles différents, allant du plus simple au plus complexe : la **Régression Logistique**, le **Random Forest** et le **XGBoost**. Ce troisième chapitre présente le fonctionnement de ces trois outils afin de comprendre comment ils transforment des données météorologiques en indices de qualité de l'air.

2. Théorie Algorithmique

La prédiction de la pollution est un défi qui dépasse les capacités de l'informatique traditionnelle. Nous entrons ici dans le domaine de l'intelligence artificielle, où la machine n'exécute plus seulement des ordres, mais extrait une connaissance à partir de l'observation du monde.

L'Intelligence artificielle est un domaine de l'informatique dont l'objectif est la simulation de l'intelligence humaine par des machines. Selon le Parlement Européen, l'IA représente « tout outil utilisé par une machine afin de reproduire des comportements liés aux humains, tels que le raisonnement, la planification et la créativité ».
Le machine learning est quant à lui une application de l'intelligence artificielle, il permet aux machines d'établir des prédictions grâce à l'expérience, les données. C'est le domaine qui donne aux machines la capacité d'apprendre sans être explicitement programmées, contrairement à la programmation classique. En effet, en programmation classique nous avons des données et des règles écrites par l'humain qui permettent de produire un résultat alors qu'avec le machine learning ce sont les données qui font les règles.
C'est cette capacité à auto-générer des règles à partir de données météorologiques historiques qui rend le Machine Learning indispensable pour prédire l'AQI dans le cadre de notre projet.

Il existe plusieurs manières de faire apprendre une machine.
Nous distinguons les cas suivants :

- **L'Apprentissage supervisé :** Elle consiste à entraîner un modèle sur des données 'étiquetées', où la réponse attendue est connue d'avance.
- **L'Apprentissage non-Supervisé :** Le modèle analyse des données non étiquetées pour y découvrir des structures ou des regroupements naturels (*clustering*).
- **L'Apprentissage semi-Supervisé :** Un mélange utilisant une petite quantité de données étiquetées et une grande quantité de données brutes.
- **L'Apprentissage par renforcement :** La machine apprend par essais et erreurs en cherchant à maximiser une récompense (souvent utilisé en robotique ou pour les jeux).

Ce mémoire se base sur l'apprentissage supervisé.

L'algorithme apprend par itération en minimisant l'écart entre sa prédiction et la réalité.
L'apprentissage supervisé permet de traiter deux grands types de problèmes : la **régression**, qui prédit une valeur numérique continue (comme la température exacte), et la **classification**, qui cherche à attribuer une donnée à une catégorie spécifique.

Dans le cadre de ce mémoire, nous ne cherchons pas à prédire une valeur brute de concentration de polluants, mais bien à situer la qualité de l'air dans un cadre sanitaire précis. Nous sommes alors dans un contexte de classification. Si de nombreux modèles d'apprentissage se limitent à une classification binaire (par exemple : l'air est-il « sain » ou « pollué » ?), notre problématique nécessite une granularité plus fine.

L'indice AQI étant structuré en six paliers de risque, nous sommes confrontés à un défi de **classification multi-classe**. Cette classification comme son nom l'indique correspond alors à une prédiction non pas d'un mais de plusieurs labels !

Les labels qu'on cherche à prédire sont les suivants :

US AQI Level		PM2.5 (µg/m³)	Health Recommendation (for 24 hour exposure)
WHO PM2.5 (µg/m³) Recommended Guidelines as of 2024: **0-5.0**			
Good	0-50	0-9.0	Air quality is satisfactory and poses little or no risk.
Moderate	51-100	9.1-35.4	Sensitive individuals should avoid outdoor activity as they may experience respiratory symptoms.
Unhealthy for Sensitive Groups	101-150	35.5-55.4	General public and sensitive individuals in particular are at risk to experience irritation and respiratory problems.
Unhealthy	151-200	55.5-125.4	Increased likelihood of adverse effects and aggravation to the heart and lungs among general public.
Very Unhealthy	201-300	125.5-225.4	General public will be noticeably affected. Sensitive groups should restrict outdoor activities.
Hazardous	301+	225.5+	General public at high risk of experiencing strong irritations and adverse health effects. Should avoid outdoor activities.

Maintenant que nous avons défini l'objectif comme une classification multi-classe pour prédire ces six niveaux de pollution, il s'agit de choisir l'outil le plus efficace.

Pour cela, nous avons décidé de tester et de comparer trois algorithmes aux fonctionnements très différents. Nous allons présenter successivement la théorie de chacun.

2.1. La Régression Logistique

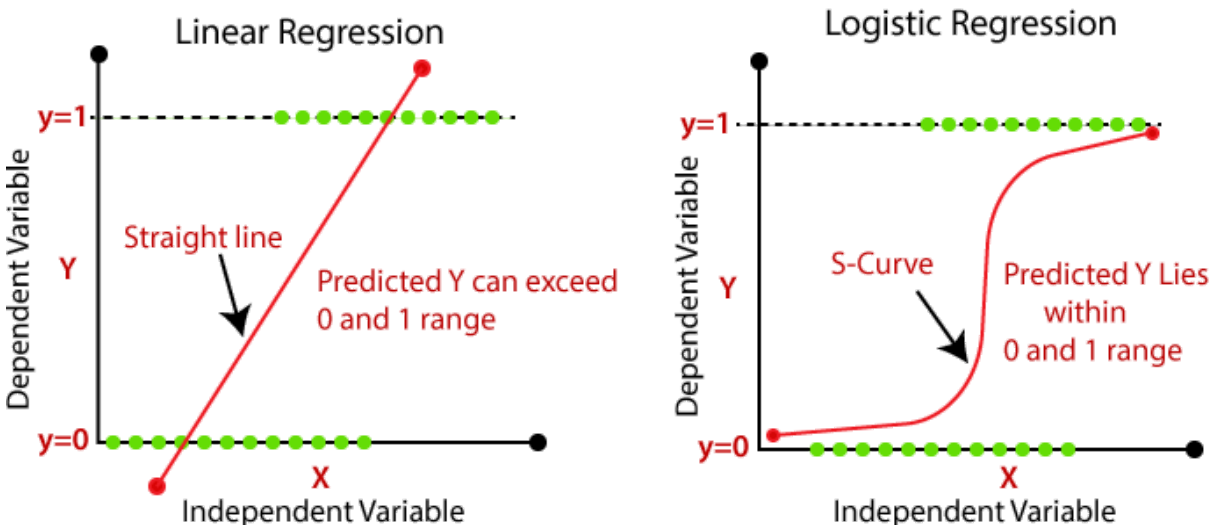
La Régression Logistique est l'un des algorithmes les plus fondamentaux et les plus utilisés en statistique et en apprentissage automatique. Malgré son nom, il s'agit bien d'un algorithme de classification et non de régression.

C'est un type de modèle linéaire qui examine la relation entre les variables prédictives (variables indépendantes X) et une variable de sortie (variable dépendante Y). Grossièrement, le modèle à pour objectif de prédire Y à partir de X.

Ce qui différencie ce modèle de la régression linéaire est la nature de la sortie X qu'il est en capacité de prédire. Contrairement à la régression linéaire qui prédit une valeur continue, la régression logistique prédit une valeur catégorielle.

Si la régression linéaire classique tente de tracer une droite passant au plus près des points, la régression logistique, elle, cherche à tracer une frontière de décision capable de séparer les classes.

Pour bien comprendre la spécificité de la régression logistique, il est utile de la comparer visuellement à la régression linéaire classique. Bien que les deux modèles partagent une base mathématique commune, leurs objectifs de prédiction diffèrent radicalement.



- **Fonctionnement de la régression linéaire :** Elle cherche à modéliser une variable quantitative en traçant une droite qui minimise la distance entre elle et tous les points du jeu de données, permettant ainsi de prédire des valeurs infinies (positives ou négatives).
- **Fonctionnement de la régression logistique :** Elle utilise une courbe spécifique pour transformer les données en probabilités, créant ainsi une 'marche' qui permet de classer les données dans des groupes distincts.

Cette distinction est cruciale pour notre étude : alors que les paramètres météorologiques sont continus, notre cible (la qualité de l'air) est organisée en paliers de risque. Pour passer de cette droite de prédiction à une classification précise, il est nécessaire d'introduire un mécanisme mathématique capable de compresser les résultats : c'est le rôle de la fonction logistique que nous allons détailler maintenant.

2.1.1. Fonctionnement Mathématique

Nous allons détailler le fonctionnement mathématiques de ce modèle. Le processus se divise en deux étapes majeures : le calcul d'un score linéaire, puis sa transformation en probabilité.

L'Équation Linéaire

Le modèle commence par fonctionner de manière identique à une régression linéaire. Il attribue un poids (β) à chaque variable météorologique en fonction de son importance constatée lors de la phase d'entraînement. Pour une observation donnée, il calcule un score combiné, souvent noté z :

$$z = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + ... + \beta_n x_n$$

Où :

- β est l'ordonnée à l'origine (le biais).
- x_1, x_2, etc. sont les valeurs de nos variables (ex. : température, vitesse du vent).
- β_1, β_2, etc. sont les coefficients (poids) appris par le modèle.

À ce stade, z peut être n'importe quel nombre réel, de $-\infty$ à $+\infty$. Il ne peut donc pas encore représenter une catégorie de pollution.

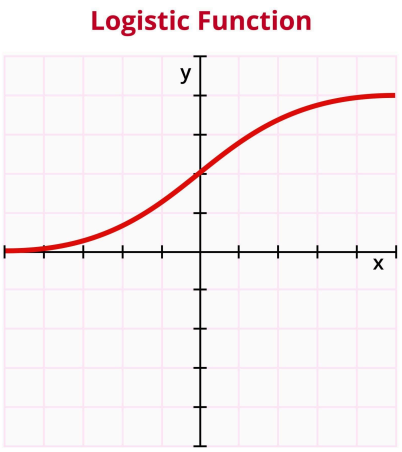
La Fonction Sigmoïde (La Probabilité)

C'est ici qu'intervient la spécificité de la régression logistique. Pour transformer ce score z en une probabilité P comprise strictement entre 0 et 1, le modèle utilise la fonction logistique, également appelée fonction sigmoïde.

Sa formule mathématique est la suivante :

$$P = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Sa représentation graphique est la suivante :



La courbe résultante possède une forme en 'S'. Quel que soit le score z calculé à l'étape précédente :

- Si z est très élevé, P se rapproche de 1 (100% de probabilité).
- Si z est très bas (négatif), P se rapproche de 0 (0% de probabilité).
- Si z est égal à 0, P est égal à 0,5.

Le Seuil de Décision et l'extension Multinomiale

Dans un cas binaire, un seuil est fixé (généralement 0,5). Si seuil $\geq$ 0,5, le modèle prédit la classe 1, sinon il prédit la classe 0.

Cependant, notre étude porte sur une classification multi-classe. Le modèle utilise alors une généralisation de la fonction sigmoïde appelée la fonction Softmax.

Au lieu de calculer une seule probabilité, le modèle calcule un score pour chaque catégorie k. La fonction Softmax compresse ces scores de sorte que chaque catégorie reçoive une probabilité proportionnelle, et que la somme de toutes les probabilités soit égale à 1 :

$$P(y = k|x) = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}$$

Le modèle choisit finalement la catégorie de pollution qui présente la probabilité la plus élevée pour les conditions météo données.

2.1.2. Fonctionnement informatique

L'implémentation informatique de la régression logistique repose sur la bibliothèque `Scikit-Learn` en Python, via la librairie `LogisticRegression`. Contrairement aux calculs manuels, l'ordinateur utilise des algorithmes d'optimisation appelés ‘solvers’ pour ajuster automatiquement les poids des variables météo afin de minimiser l'erreur de classement. Pour garantir l'efficacité du code, les données sont d'abord standardisées (mises à la même échelle), puis traitées via une stratégie multinomiale (ou Softmax) qui permet de gérer simultanément les 6 catégories de pollution de l'AQI en attribuant à chaque ligne de données la classe ayant la probabilité la plus élevée.

Un exemple d’utilisation du modèle de Régression Logistique :

```python
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
```

```
# 1. Préparation du modèle
model = Pipeline(steps=[
    ('scaler', StandardScaler()), # Standardise les données (moyenne=0,
écart-type=1)
    ('clf', LogisticRegression(
        max_iter=3000,           # Nombre d'itérations
        multi_class='auto',      # Gère le binaire ou le multi-classe
automatiquement
        class_weight='balanced', # Gère le déséquilibre entre les classes
        random_state=42
    ))
])

# 2. Entraînement
# X_train : nos variables explicatives (features)
# y_train : nos étiquettes (labels)
model.fit(X_train, y_train)

# 3. Prédiction
# On applique le modèle sur des données jamais vues
y_pred = model.predict(X_test)
# 4. Évaluation
print(f'Accuracy globale : {accuracy_score(y_test, y_pred):.4f}')
print('\nRapport détaillé des performances :')
print(classification_report(y_test, y_pred))
```

2.2. Le Random Forest

Le Random Forest est un algorithme d'apprentissage supervisé utilisé tant pour la classification que pour la régression. Il combine les résultats de plusieurs arbres de décision pour obtenir un résultat unique.

2.2.1. Fonctionnement Mathématique

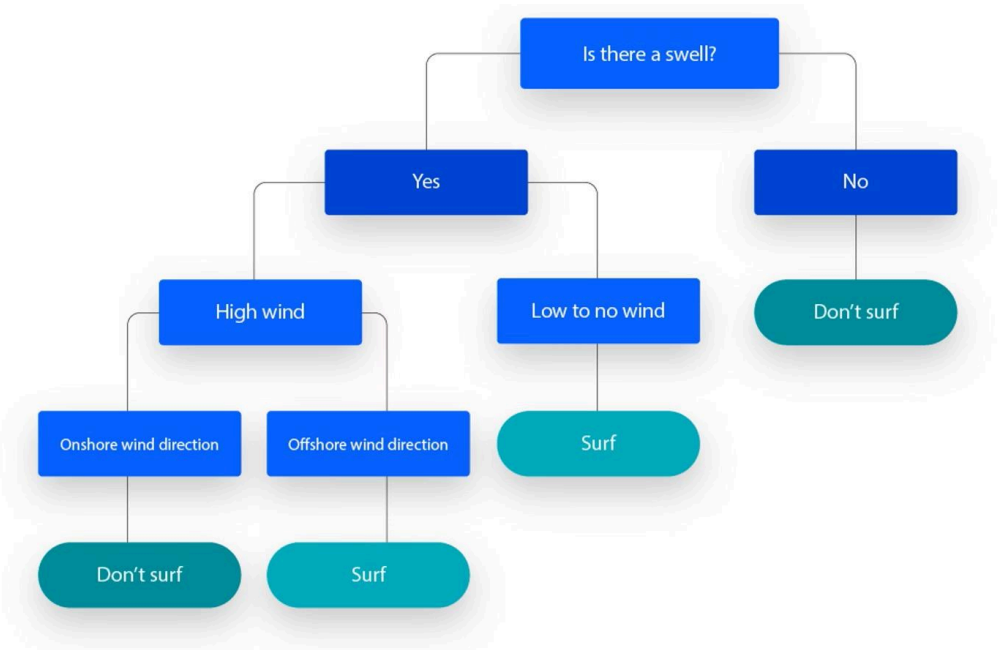
Pour comprendre le Random Forest, il faut d'abord définir son unité de base : l'arbre de décision.

L'Arbre de Décision

Un arbre de décision est un modèle qui classe les données en suivant une série de règles logiques de type 'Si... Alors...'. Sa structure ressemble à un arbre inversé :

- **Le nœud racine :** La première question posée sur la variable la plus importante.
- **Les nœuds internes :** Des questions successives qui divisent les données en deux branches.
- **Les feuilles :** Les extrémités de l'arbre qui donnent la réponse finale.

Voici un exemple graphique d'un arbre de décision :



Pour savoir quelle question poser à chaque étape (quelle variable choisir et quel seuil), l'algorithme utilise un critère de décision : l'**Indice de Gini**[11].

$$G = 1 - \sum_{i=1}^{n} P_i^2$$

Où P_i est la proportion d'éléments appartenant à la classe i dans le nœud, et n le nombre de classes (6 dans notre projet).

Le principe de la Forêt (Ensemble Learning)

Un seul arbre a tendance à faire du 'sur-apprentissage' (*overfitting*) : il apprend par cœur les données d'entraînement mais ne sait pas généraliser.

[11] Dans le cadre du Machine Learning, l'**indice de Gini** est une mesure de l'impureté (ou du désordre) d'un groupe de données. Il permet à l'algorithme de choisir la variable (ex. : Température) et le seuil (ex. : > 25°C) qui diviseront le mieux les observations en groupes les plus homogènes possibles.

- **Un indice de Gini égal à 0** signifie que le groupe est 'pur' : toutes les données appartiennent à la même catégorie (par exemple, toutes les journées sont 'Good AQI').
- **Plus l'indice est élevé**, plus le mélange entre les différentes catégories de pollution est important dans ce groupe.

L'objectif de l'arbre de décision est de minimiser cet indice à chaque étape pour obtenir les prédictions les plus nettes possibles.

C'est ici qu'intervient le Random Forest. Il utilise une approche d'Ensemble Learning (apprentissage d'ensemble), qui consiste à combiner les prédictions de centaines d'arbres différents pour obtenir une décision finale plus robuste et plus précise.

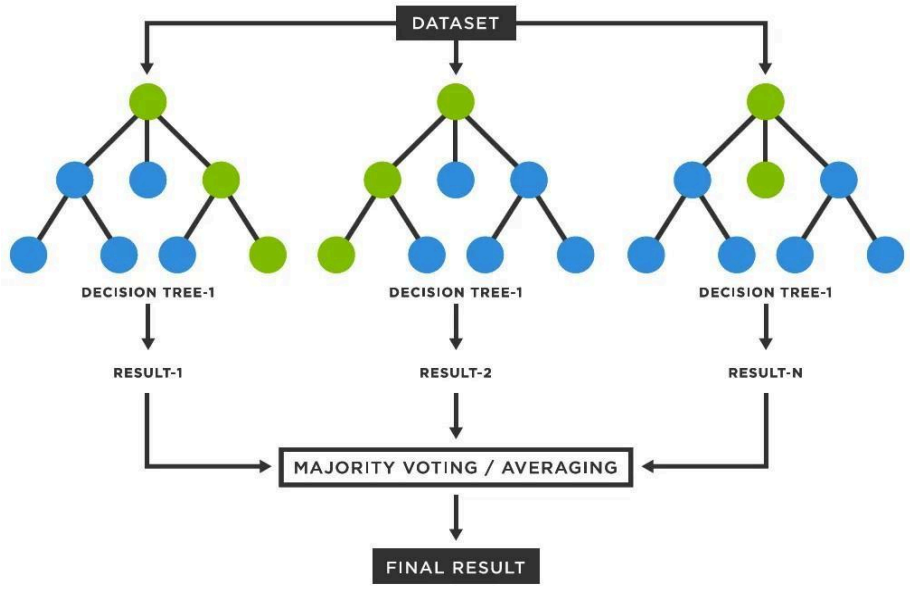
Le Random Forest utilise deux mécanismes :

- **Le Bootstrapping :** Au lieu d'utiliser tout le jeu de données pour entraîner chaque arbre, l'algorithme crée des sous-ensembles aléatoires. Pour chaque arbre, il tire au sort (avec remplacement) un échantillon de données. Cela signifie qu'un arbre peut 'voir' plusieurs fois la même ligne de données, tandis qu'un autre ne la verra jamais.
- **La sélection aléatoire de variables :** Lors de la construction de chaque nœud au sein d'un arbre, l'algorithme ne cherche pas la meilleure division parmi la totalité des variables explicatives disponibles. Il sélectionne aléatoirement un sous-ensemble restreint de variables (caractéristiques) et ne choisit la règle de décision que parmi celles-ci. Ce mécanisme est crucial : il empêche les variables les plus prédictives de dominer systématiquement la structure de tous les arbres. En forçant les arbres à explorer des relations moins évidentes entre les données, on garantit une grande diversité au sein de la forêt, ce qui renforce la robustesse globale du modèle.

Une fois que la forêt, composée de plusieurs centaines d'arbres indépendants, a terminé sa phase d'entraînement, elle est prête à réaliser des prédictions sur de nouvelles données. Le processus de décision finale suit une logique de consensus :

1. **Prédiction individuelle :** Chaque arbre de la forêt analyse la donnée d'entrée et émet son propre vote en faveur d'une classe.
2. **Agrégation :** L'algorithme comptabilise l'ensemble des votes émis par tous les arbres.
3. **Verdict final :** La classe ayant obtenu la majorité absolue ou relative des suffrages est retenue comme la prédiction finale du modèle.

Sa représentation graphique :



D'un point de vue statistique, cette combinaison de modèles diversifiés permet de réduire drastiquement la variance globale (le risque d'erreur lié à la sensibilité aux données d'entraînement) sans augmenter le biais (l'incapacité à capturer la complexité du réel). C'est cette synergie qui fait du Random Forest l'un des algorithmes les plus fiables et les plus performants pour les problèmes de classification complexe.

2.2.2. Fonctionnement informatique

L'implémentation du Random Forest en Python se fait via la librairie `RandomForestClassifier` de la bibliothèque `Scikit-Learn`. Contrairement à la régression logistique, cet algorithme est non paramétrique : il ne cherche pas à résoudre une équation mathématique globale, mais construit une structure de données (les arbres) en mémoire vive.

Le comportement du modèle informatique est dicté par plusieurs paramètres que le développeur doit ajuster :

- `n_estimators` : C'est le nombre d'arbres dans la forêt (généralement entre 100 et 500). Plus ce nombre est élevé, plus le modèle est robuste, mais plus il consomme de mémoire et de temps de calcul.
- `max_depth` : La profondeur maximale de chaque arbre. On la limite souvent pour forcer les arbres à rester simples et éviter le sur-apprentissage.
- `max_features` : Le nombre de variables sélectionnées aléatoirement à chaque nœud (souvent égal à la racine carrée du nombre total de variables).

L'un des grands avantages informatiques du Random Forest est sa capacité à être parallélisé. Comme chaque arbre est construit indépendamment des autres (grâce au bootstrapping), l'ordinateur peut distribuer le calcul sur tous les cœurs du processeur simultanément.

Un exemple d'utilisation du modèle de Random Forest :

```python
from sklearn.ensemble import RandomForestClassifier

# Initialisation du modèle avec des paramètres optimisés
rf_model = RandomForestClassifier(
    n_estimators=100,      # Nombre d'arbres
    max_depth=10,          # Limite la complexité des arbres
    max_features='sqrt',   # Sélection aléatoire des variables
    n_jobs=-1,             # Utilise toute la puissance du processeur
    random_state=42
)

# Entraînement : construction de la forêt en mémoire
rf_model.fit(X_train, y_train)

# Chaque arbre vote pour rendre une décision finale
predictions = rf_model.predict(X_test)
```

```
# Analyse de l'importance des variables
importances = rf_model.feature_importances_
```

Un atout majeur de l'implémentation informatique est l'accès à l'attribut `feature_importances_`. Le modèle calcule automatiquement le score de chaque variable en fonction de sa capacité à réduire l'indice de Gini à travers tous les arbres de la forêt. Cela permet d'identifier quelles données ont eu le plus d'impact sur la décision finale.

2.3. Le XGBoost

Le XGBoost est un modèle qui utilise des arbres de décision auxquels est appliqué le boosting de gradient, ce dernier étant un algorithme de boosting d'apprentissage supervisé qui utilise la descente de gradient. Il est connu pour sa vitesse, son efficacité et sa capacité à s'adapter à de grands jeux de données.

2.3.1. Fonctionnement Mathématique

Contrairement au Random Forest où les arbres sont indépendants, le XGBoost construit ses arbres les uns après les autres.

L'Apprentissage Résiduel (Correction d'erreurs)

Le processus commence par un modèle très simple (souvent une simple moyenne). L'algorithme calcule ensuite l'écart entre la réalité et sa prédiction : c'est ce qu'on appelle le résidu (l'erreur). Le deuxième arbre est alors entraîné non pas pour prédire la cible finale, mais pour prédire l'erreur du premier arbre. Le troisième arbre corrige l'erreur du second, et ainsi de suite.

La Descente de Gradient

Pour minimiser l'erreur globale, XGBoost utilise une méthode mathématique appelée la Descente de Gradient. Il calcule la pente (le gradient) de la fonction de perte pour savoir dans quelle 'direction' ajuster les prédictions afin d'atteindre l'erreur minimale.

La Régularisation (Le secret de XGBoost)

Ce qui rend le XGBoost 'Extreme', c'est l'ajout d'une pénalité mathématique dans sa fonction de coût. Il pénalise les arbres trop complexes ou trop profonds. Cela permet de garder un modèle fluide et d'éviter l'overfitting.

L'équation est la suivante :

$$\mathcal{L}(\phi) = \sum_i l(\hat{y}_i, y_i) + \sum_k \Omega(f_k)$$

- $l(\hat{y}_i, y_i)$: La fonction de perte (ex: erreur quadratique) qui calcule l'écart entre la prédiction et la réalité.
- $\Omega(f_k)$: Le terme de régularisation qui pénalise la complexité des arbres pour éviter le sur-apprentissage.

2.3.2. Fonctionnement informatique

Informatiquement, `XGBoost` est une bibliothèque séparée de `Scikit-Learn`, conçue pour être extrêmement rapide et optimisée pour la mémoire.

Les hyperparamètres spécifiques de contrôle sont principalement :

- `learning_rate` : C'est le 'pas' de l'apprentissage. S'il est petit, le modèle apprend lentement mais plus sûrement.
- `n_estimators` : Le nombre total d'arbres (étapes de correction).

`XGBoost` gère nativement les données manquantes (il apprend tout seul quel chemin prendre si une donnée est absente) et utilise le calcul parallèle au niveau des processeurs pour construire les arbres beaucoup plus vite que le boosting classique.

Un exemple d'utilisation du modèle XGBoost :

```python
from xgboost import XGBClassifier

# Initialisation du modèle XGBoost
xgb_model = XGBClassifier(
    n_estimators=100,
    learning_rate=0.1,    # Vitesse d'apprentissage
    max_depth=6,          # Profondeur des arbres
    use_label_encoder=False,
    eval_metric='mlogloss', # Fonction de perte pour le multi-classe
    random_state=42
)

# Entraînement séquentiel
xgb_model.fit(X_train, y_train)

# Prédiction
y_pred = xgb_model.predict(X_test)
```

Ce chapitre nous a permis de poser les fondements théoriques nécessaires à la compréhension de notre démarche. Nous avons établi que la prédiction de la qualité de l'air relève de l'apprentissage supervisé et, plus précisément, de la classification multi-classe.

En explorant trois approches distinctes, nous avons identifié des forces complémentaires :

- La **Régression Logistique** nous offre une base de référence simple et interprétable.
- Le **Random Forest** apporte de la robustesse grâce à la diversité d'une forêt d'arbres et au vote majoritaire.
- Le **XGBoost** pousse la performance à son maximum en corrigeant séquentiellement ses erreurs via le gradient boosting.

La compréhension de ces mécanismes mathématiques et informatiques est essentielle, mais elle n'est que la première étape. Pour transformer ces concepts en un outil de prédiction capable d'aider concrètement les populations, nous devons maintenant confronter ces algorithmes à la réalité du terrain.

Dans la partie suivante, nous allons détailler la mise en œuvre technique de notre projet. Nous passerons de la théorie à la pratique : de la récupération et du nettoyage des données météo mondiales jusqu'à l'entraînement final de nos modèles et l'analyse de leurs performances réelles. C'est ici que nous déterminerons lequel de nos trois candidats est le plus apte à devenir le moteur de notre solution de prédiction de l'AQI.

3. Application et Développement — Le cœur technique

Ici, l’étape cruciale, la mise en application.

Dans cette partie, nous présenterons notre travail de data science effectué afin de répondre à la problématique, qui est, pour rappel : 'Modélisation prédictive de la qualité de l'air : dans quelle mesure le Machine Learning permet-il de prédire la qualité de l'air à partir des seules données météorologiques, indépendamment des mesures directes de polluants ?'.

3.1. Le choix des technologies et langages

3.1.1. L’IDE

Nous avons choisi d’effectuer ce travail avec l’IDE ‘VS CODE’, un éditeur de code source gratuit et multiplateforme développé par Microsoft, qui permet d’utiliser de très nombreux langages.

Nous avons notamment utilisé l'extension Jupyter, qui permet d'intégrer des Notebooks directement dans l'interface. Cet environnement a facilité l'exploration interactive des données, nous permettant d'exécuter des blocs de code de manière isolée et de les visualiser instantanément.

Son terminal intégré a permis le versionning GIT de notre travail mais également le lancement des serveurs locaux nécessaires au test de notre application web.

3.1.2. Les langages

Pour mener à bien ce projet, de la manipulation des données brutes jusqu'à la création d'une interface utilisateur, nous nous sommes appuyés sur deux langages complémentaires.

Python : Le moteur de l'analyse et de l'intelligence

Python s'est imposé comme le choix naturel pour ce projet. C'est aujourd'hui le langage de référence en data science en raison de son immense écosystème de bibliothèques spécialisées. Dans notre étude, Python intervient à trois niveaux cruciaux :

- **Le traitement de données :** Grâce à des librairies comme `Pandas` pour la manipulation de tableaux de données et `NumPy` pour les calculs mathématiques, nous avons pu nettoyer et transformer un jeu de données de plus de 120 000 lignes.
- **La modélisation prédictive :** Nous avons utilisé la librairie `Scikit-Learn` (pour la Régression Logistique et le Random Forest) ainsi que la librairie `XGBoost`. Ces outils permettent non seulement d'entraîner les modèles, mais aussi de valider leur précision via des tests statistiques rigoureux.
- **Le pilotage de l'application :** Au-delà de l'analyse, Python sert également de 'back-end' pour notre application web, faisant le pont entre les entrées de l'utilisateur et le modèle stocké pour générer une prédiction en temps réel.

HTML : L'interface et la restitution

Si Python constitue le 'cerveau' du projet, le **HTML** (HyperText Markup Language) en est le 'visage'.

- **Structure de l'application :** Nous avons utilisé le HTML pour structurer l'interface de notre application web. Il permet de définir les formulaires de saisie où l'utilisateur entre ses paramètres météorologiques et d'organiser l'affichage des résultats de prédiction.
- **Expérience utilisateur :** Couplé à Python (via le framework `Streamlit`), le HTML permet de transformer un script de calcul complexe en un outil interactif, visuel et accessible à un utilisateur non-spécialiste, facilitant ainsi la compréhension des enjeux de la qualité de l'air.

3.2. Les librairies

3.2.1. L'importation des librairies

Dans l'écosystème Python, une librairie (ou bibliothèque) est un ensemble de fonctions et de méthodes pré-écrites que les développeurs peuvent importer pour effectuer des tâches spécifiques sans avoir à réécrire le code de zéro. Pour ce projet, nous avons fait appel à plusieurs librairies. Elles nous ont permis de manipuler les données, de les visualiser et de construire nos modèles prédictifs.

Voici la liste des librairies importées dans notre notebook :

```python
import pandas as pd
import numpy as np
import seaborn as sns
import textwrap
import joblib
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec
import xgboost as xgb
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import StratifiedKFold, cross_validate,
train_test_split
from sklearn.metrics import make_scorer, accuracy_score,
precision_score, recall_score, f1_score, roc_auc_score, roc_curve, auc
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler, label_binarize
from sklearn.linear_model import LogisticRegression
```

Description des librairies :

- **`Pandas(pd)`:** Librairie incontournable pour la manipulation et l'analyse de données tabulaires. Elle nous a permis de charger notre fichier CSV, de filtrer les colonnes, de traiter les dates et de structurer nos données sous forme de dataframes.
- **`NumPy(np)`:** Dédiée au calcul scientifique permettant de manipuler des tableaux de données volumineux et d'exécuter des opérations mathématiques optimisées.
- **`Seaborn(sns)`:** Extension de `Matplotlib` spécialisée dans les graphiques statistiques avancés, employée pour générer la matrice de corrélation de notre jeu de données.
- **`Textwrap`:** Utilisée pour formater, envelopper ou tronquer les blocs de texte longs.
- **`Joblib`:** Permet de sauvegarder le modèle entraîné au format .pkl pour le charger instantanément dans l'application sans réentraînement.
- **`Matplotlib(plt)`:** Outil de base pour la création de visualisations statiques en deux dimensions, utilisé ici pour analyser la distribution de nos variables.
- **`XGBoost(xgb)`:** Librairie contenant l'algorithme 'eXtreme Gradient Boosting'. Elle est réputée pour sa très haute performance et sa rapidité.
- **`Scikit-Learn(sklearn)`:** Librairie de référence pour le Machine Learning classique.

Ces librairies avant d'être importées 'import x', elles doivent être installées dans notre environnement 'pip install x'. Pour ce faire, il est une bonne pratique de créer un environnement virtuel.

3.2.2. L'environnement virtuel

Pour exécuter du code nous avons besoin d'être dans un environnement. Cependant, en installant tout dans l'environnement global du système, on s'expose à une instabilité où la mise à jour d'un package pour un projet pourrait rendre un autre projet totalement inutilisable !

C'est là que l'environnement virtuel intervient. C'est un outil qui permet de conserver les dépendances requises par différents projets dans des emplacements séparés.

Pour ce projet nous avons alors créé et utilisé un environnement virtuel nommé `venv`.

3.3. Le jeu de données

3.3.1. Contexte

Pour effectuer notre analyse, il nous fallait un jeu de données qui regroupe les caractéristiques suivantes :

- des variables météorologiques
- des variables de polluants
- un nombre de lignes assez conséquent pour entraîner au mieux notre modèle

Nous avons trouvé sur Kaggle, une plateforme qui fournit des jeux de données, un Dataset qui correspondait à nos critères.

Le Dataset est le suivant :
https://www.kaggle.com/datasets/nelgiriyewithana/global-weather-repository

Ce jeu de données fournit des informations météorologiques quotidiennes pour les capitales du monde entier.

Ce jeu de données est particulièrement précieux pour analyser les modèles météorologiques mondiaux, explorer les tendances climatiques et comprendre les relations entre les différents paramètres météorologiques.

Nous le téléchargeons de Kaggle et le chargeons dans notre notebook, il s'agit d'un fichier CSV.

```python
df = pd.read_csv('../Dataset/GlobalWeatherRepository.csv')
```

3.3.2. Exploration

Une fois notre jeu de données chargé, nous pouvons procéder à son exploration.

Cette étape permet d'obtenir un premier aperçu technique de la structure des données brutes afin d'en évaluer le potentiel. L'objectif est d'identifier précisément les variables à notre disposition, de détecter la présence de valeurs manquantes et de vérifier la cohérence des types de données pour déterminer les transformations nécessaires avant le nettoyage.

Les commandes principales utilisées pour faire ceci ont été :

```python
# connaître les dimensions du jeu de données
df.shape

# audit technique des données
df.info()

# détection des valeurs nulles
df.isna().sum().sort_values(ascending=False)
```

Grâce à l'exécution de ces commandes nous avons pu obtenir les informations suivantes :

- Le jeu de données est composé de 126476 lignes et 41 colonnes.
- Nous n'avons aucune valeur nulle dans ce Dataset !
- Les noms des colonnes
- Les types des colonnes : object, float64, int64

Ci-dessous, la sortie de la commande `df.info()`:

```python
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 126476 entries, 0 to 126475
Data columns (total 41 columns):
 #   Column                        Non-Null Count   Dtype
---  ------                        --------------   -----
 0   country                       126476 non-null  object
 1   location_name                 126476 non-null  object
 2   latitude                      126476 non-null  float64
 3   longitude                     126476 non-null  float64
 4   timezone                      126476 non-null  object
 5   last_updated_epoch            126476 non-null  int64
 6   last_updated                  126476 non-null  object
 7   temperature_celsius           126476 non-null  float64
 8   temperature_fahrenheit        126476 non-null  float64
 9   condition_text                126476 non-null  object
 10  wind_mph                      126476 non-null  float64
 11  wind_kph                      126476 non-null  float64
 12  wind_degree                   126476 non-null  int64
 13  wind_direction                126476 non-null  object
 14  pressure_mb                   126476 non-null  float64
 15  pressure_in                   126476 non-null  float64
 16  precip_mm                     126476 non-null  float64
 17  precip_in                     126476 non-null  float64
 18  humidity                      126476 non-null  int64
 19  cloud                         126476 non-null  int64
 20  feels_like_celsius            126476 non-null  float64
 21  feels_like_fahrenheit         126476 non-null  float64
 22  visibility_km                 126476 non-null  float64
 23  visibility_miles              126476 non-null  float64
 24  uv_index                      126476 non-null  float64
 25  gust_mph                      126476 non-null  float64
 26  gust_kph                      126476 non-null  float64
 27  air_quality_Carbon_Monoxide   126476 non-null  float64
 28  air_quality_Ozone             126476 non-null  float64
 29  air_quality_Nitrogen_dioxide  126476 non-null  float64
 30  air_quality_Sulphur_dioxide   126476 non-null  float64
 31  air_quality_PM2.5             126476 non-null  float64
 32  air_quality_PM10              126476 non-null  float64
 33  air_quality_us-epa-index      126476 non-null  int64
 34  air_quality_gb-defra-index    126476 non-null  int64
 35  sunrise                       126476 non-null  object
 36  sunset                        126476 non-null  object
 37  moonrise                      126476 non-null  object
 38  moonset                       126476 non-null  object
 39  moon_phase                    126476 non-null  object
 40  moon_illumination             126476 non-null  int64
dtypes: float64(23), int64(7), object(11)
memory usage: 39.6+ MB
```

Nous avons alors identifié 4 types de variables :

- les variables géographiques
- les variables météorologiques
- les variables ‘polluants’
- les variables temporelles

Nous avons également identifié notre variable cible. Nous avons deux variables qui peuvent être utilisées comme cible :

- `air_quality_us-epa-index` : mesure de la qualité de l’air selon l’échelle de l’US EPA (United States Environmental Protection Agency). Les valeurs vont de 1 à 6, correspondant à des catégories allant de 'Good' à 'Hazardous'.
- `air_quality_gb-defra-index` : mesure équivalente selon l’échelle britannique DEFRA. Les valeurs se situent entre 1 et 10, un numéro plus élevé indiquant une mauvaise qualité de l’air.

Pour rappel, ces index de qualité de l’air sont des indicateurs simplifiés permettant de traduire des concentrations complexes de polluants chimiques en une échelle de risque sanitaire compréhensible par le grand public.

Pour faciliter la communication immédiate du risque, chaque niveau de ces indices est associé à un code couleur universel. Nous avons pris en compte cette représentation par couleur dans notre travail, nous l'utiliserons pour nos visualisations :

Références des indices de qualité de l'air

US EPA Index

Valeur	Catégorie	Couleur
1	Good	#00e400
2	Moderate	#ffff00
3	Unhealthy for Sensitive Groups	#ff7e00
4	Unhealthy	#ff0000
5	Very Unhealthy	#8f3f97
6	Hazardous	#7e0023

GB DEFRA Index

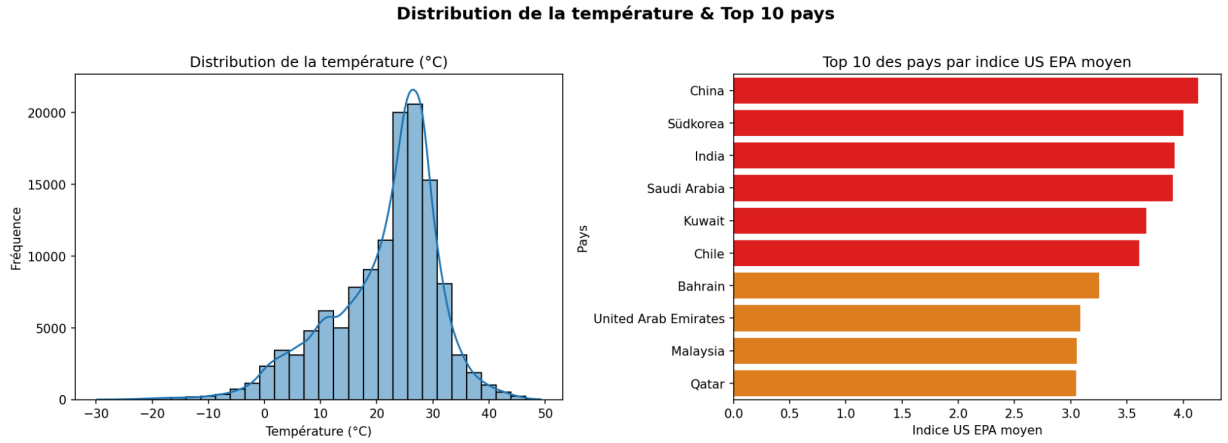
Valeur	Catégorie	Couleur
1	Low	#00e400
2	Low	#00e400
3	Low	#00e400
4	Moderate	#ffff00
5	Moderate	#ffff00
6	Moderate	#ff7e00
7	High	#ffb3d9
8	High	#ff0000
9	High	#7e0023
10	Very High	#8f3f97

Nous ferons le choix final de la variable cible dans la partie de pré-traitement.

3.3.3. Visualisations

Avec les librairies `Matplotlib` et `Seaborn`, nous avons alors pu générer des visualisations permettant de mieux comprendre le jeu de données. L'objectif est d'explorer le jeu de données avant nettoyage pour en dégager les premières tendances.

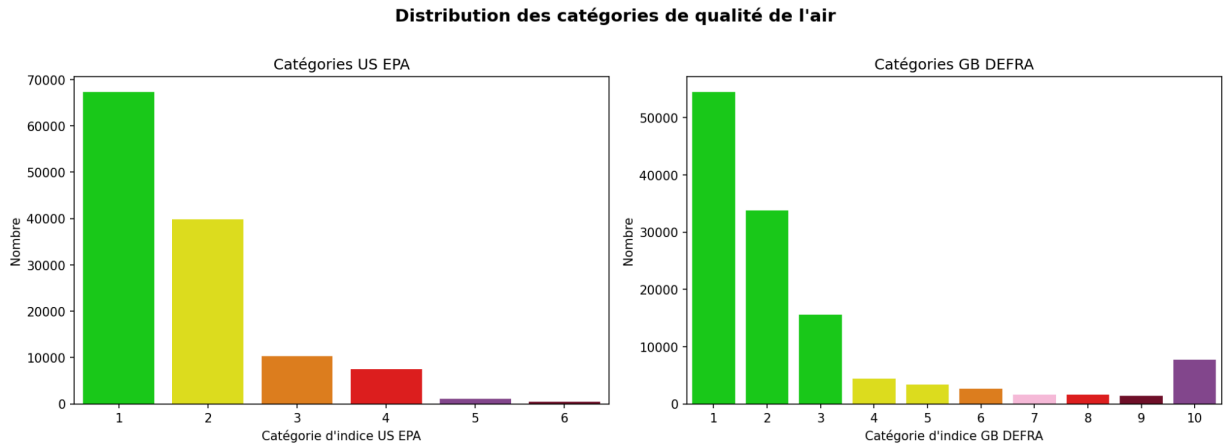
Les visualisations sont les suivantes :



→ Distribution de la température : La majorité des relevés mondiaux se concentre entre 20°C et 30°C, révélant un jeu de données riche en climats tempérés et tropicaux avec très peu de valeurs extrêmes négatives.

→ Top 10 des pays par AQI : La Chine, la Corée du Sud et l'Inde dominent ce classement avec un indice moyen proche de 4, signalant une qualité de l'air régulièrement 'mauvaise' selon les standards US EPA.

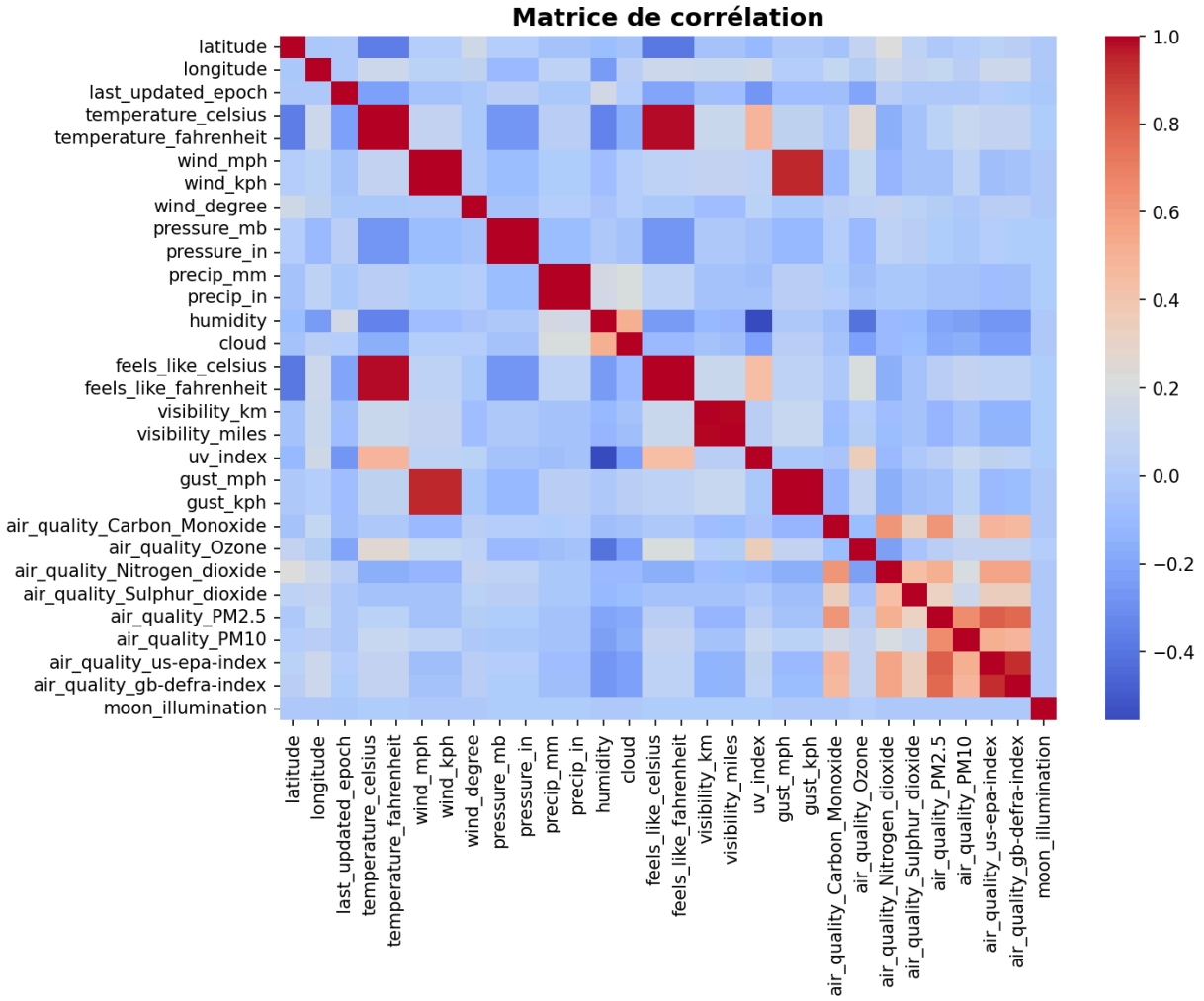
—



→ Distribution US EPA : La grande majorité des relevés (niveaux 1 et 2) indique un air sain ou modéré, tandis que les catégories critiques (niveaux 5 et 6) restent très marginales dans le jeu de données.

→ Distribution GB DEFRA : On observe une tendance similaire avec une prédominance du niveau 1 ('Low'), mais l'échelle plus fine révèle un pic inattendu au niveau 10 ('Very High'), suggérant des épisodes de pollution extrême localisés.

—



Une matrice de corrélation est un outil statistique qui permet de mesurer l'intensité et la direction de la relation entre plusieurs variables numériques prises deux à deux. Dans le cadre de ce projet, elle nous sert à comprendre comment les facteurs météorologiques interagissent entre eux et avec les polluants atmosphériques.

→ Corrélations fortes et redondances : On observe des blocs rouges intenses (corrélation proche de 1) entre les mesures en Celsius et Fahrenheit, ou entre les km/h et mph pour le vent. Cela confirme la nécessité de supprimer ces doublons lors du nettoyage pour éviter la multicolinéarité.

→ Lien météo et polluants : Il existe une corrélation visuelle notable entre l'indice UV (`uv_index`) et certains polluants comme l'Ozone, ainsi qu'entre la visibilité et les particules fines (PM2.5/PM10), validant le potentiel prédictif de la météo sur la qualité de l'air.

—

L’analyse exploratoire a permis de confirmer la richesse et l’intégrité du jeu de données, tout en révélant les premières dynamiques entre les paramètres environnementaux. Cependant, la présence de variables redondantes et de formats bruts nécessite une phase de préparation rigoureuse. L'enjeu est désormais de transformer ce matériau complexe en un ensemble de données optimisé et pertinent pour garantir la performance des futurs algorithmes de prédiction.

3.4. Pré-traitement des données

Le pré-traitement des données est l'étape intermédiaire entre la collecte des données brutes et l'entraînement du modèle de Machine Learning. Il s'agit d'un processus de raffinage visant à corriger, filtrer et transformer les données afin d'éliminer le 'bruit' (informations inutiles ou contradictoires) qui pourrait induire l'algorithme en erreur.

Cette étape est cruciale car la qualité des prédictions d'un modèle dépend directement de la qualité des données en entrée (principe du *'Garbage In, Garbage Out*[12]*'*).

Le pré-traitement sert principalement à :

- **Améliorer la précision :** En ne conservant que les variables qui ont un réel pouvoir prédictif.
- **Réduire le temps de calcul :** En supprimant les colonnes redondantes qui alourdissent inutilement les opérations mathématiques.
- **Standardiser les formats :** Pour rendre les données interprétables par les fonctions mathématiques des bibliothèques comme `Scikit-Learn`.

3.4.1. Les Opérations et choix sur les données

Dans cette partie, nous renseignerons toutes les opérations qu’on a opérées sur les données ainsi que les choix qu’on a fait, en les justifiant.

Suppression de variables

Nous avons procédé à l'élimination des colonnes jugées non nécessaires pour notre futur modèle de prédiction, afin de simplifier le jeu de données et de se concentrer uniquement sur les variables explicatives les plus pertinentes.

```python
cols_to_drop = ['latitude', 'longitude','last_updated_epoch',
'condition_text', 'temperature_fahrenheit', 'wind_mph', 'pressure_in',
```

[12] **Garbage In, Garbage Out (GIGO) :** Concept informatique stipulant que la qualité de la sortie d'un système dépend de la qualité de ses entrées. Dans le cadre du Machine Learning, cela signifie qu'un modèle entraîné sur des données de mauvaise qualité produira des prédictions erronées, indépendamment de sa complexité algorithmique.

```
                'precip_in', 'visibility_miles', 'gust_mph',
'moon_phase', 'moon_illumination', 'sunrise', 'sunset', 'moonrise',
'moonset',
                'feels_like_fahrenheit']
df_2 = df.drop(columns=cols_to_drop)
```

Nous avons également supprimé la variable `air_quality_gb-defra-index`.

```python
df_2 = df_2.drop(columns=['air_quality_gb-defra-index'])
```

Pourquoi ce choix ?

Pour rappel, nous avions deux variables cibles : `air_quality_gb-defra-index` et `air_quality_us-epa-index`.

Nous avons choisi de retenir l'indice US EPA comme cible finale pour notre modèle pour deux raisons principales.

D'une part, l'échelle américaine en six catégories est plus synthétique et équilibrée pour une analyse à l'échelle mondiale. D'autre part, la distribution des données montrait une concentration plus cohérente sur ces six classes, évitant la trop grande dispersion que l'on pouvait observer sur les dix niveaux de l'échelle britannique. En supprimant l'indice DEFRA, nous éliminons ainsi toute redondance et forçons le modèle à se concentrer sur une unique échelle de classification, rendant l'apprentissage plus robuste et l'interprétation des résultats plus claire.

Formatage des variables

Nous avons modifié le type de certaines colonnes pour les faire correspondre à leur nature réelle, comme le passage des variables textuelles en `category` et des dates en `datetime`. Cette étape est essentielle pour optimiser la mémoire et garantir un traitement statistique cohérent lors de la modélisation.

```python
df_2['air_quality_us-epa-index'] =
df_2['air_quality_us-epa-index'].astype('category')
df_2['wind_direction'] = df_2['wind_direction'].astype('category')
df_2['last_updated'] = pd.to_datetime(df_2['last_updated'])
df_2['country'] = df_2['country'].astype('category')
df_2['location_name'] = df_2['location_name'].astype('category')
```

```
df_2['timezone'] = df_2['timezone'].astype('category')
```

- Le type `datetime` **:** La colonne `last_updated` était interprétée comme une simple chaîne de caractères. En la convertissant en `datetime`, nous transformons ce texte en un objet temporel. Cela permet à Python de comprendre la chronologie et nous donne la possibilité d'extraire mécaniquement des informations précises comme l'heure ou le mois, ce qui est indispensable pour analyser les variations cycliques de la pollution.
- Le type `category` **:** Nous avons appliqué ce type aux variables qualitatives (pays, villes, directions du vent et index cible). Contrairement au type `object` qui traite chaque cellule comme du texte indépendant, le type `category` stocke les données de manière plus efficace en mémoire en ne conservant qu'une seule fois chaque étiquette unique associée à un code numérique interne. Pour les modèles de Machine Learning, cela facilite la distinction entre les classes et accélère les calculs, tout en garantissant que les index de qualité de l'air soient traités comme des paliers discrets et non comme des valeurs continues infinies.

Modification d'une variable

Pendant l'exploration, nous avons constaté que la variable `timezone` était composée de manière suivante : '`continent/city`'.

```python
# Exploration de la colonne timezone
df['timezone'].head(5)
```

```python
0                        Asia/Kabul
1                      Europe/Tirane
2                     Africa/Algiers
3                     Europe/Andorra
4                      Africa/Luanda
Name: timezone, dtype: object
```

Nous avons alors modifié la variable `timezone` pour ne conserver que l'information liée au continent. La précision de la ville étant déjà présente dans la colonne `location_name`, cette transformation permet d'éliminer une redondance d'information tout en créant une variable catégorielle plus large et plus utile.

```python
# ne garder que le continent dans timezone (avant le '/')
```

```
df_2['timezone'] = df_2['timezone'].astype(str).str.split('/').str[0]
# éliminer les éventuels espaces
df_2['timezone'] = df_2['timezone'].str.strip()
```

Renommage des variables

Nous avons renommé les variables ci-dessous afin d'attribuer des noms plus explicites et compréhensibles, reflétant précisément le contenu réel des données après nos premières transformations.

```python
df_2 = df_2.rename(columns={
    'timezone': 'continent',
    'location_name': 'city',
    'last_updated': 'date'
})
```

Création de variables

Nous avons procédé ici à la création de nouvelles variables, notamment par l'extraction d'unités temporelles (année, mois, jour, heure) à partir de la variable date. Cette étape est essentielle pour enrichir notre jeu de données, car elle nous permettra d'effectuer des comparaisons temporelles plus fines et d'identifier des saisonnalités ou des cycles quotidiens dans l'évolution de la pollution.

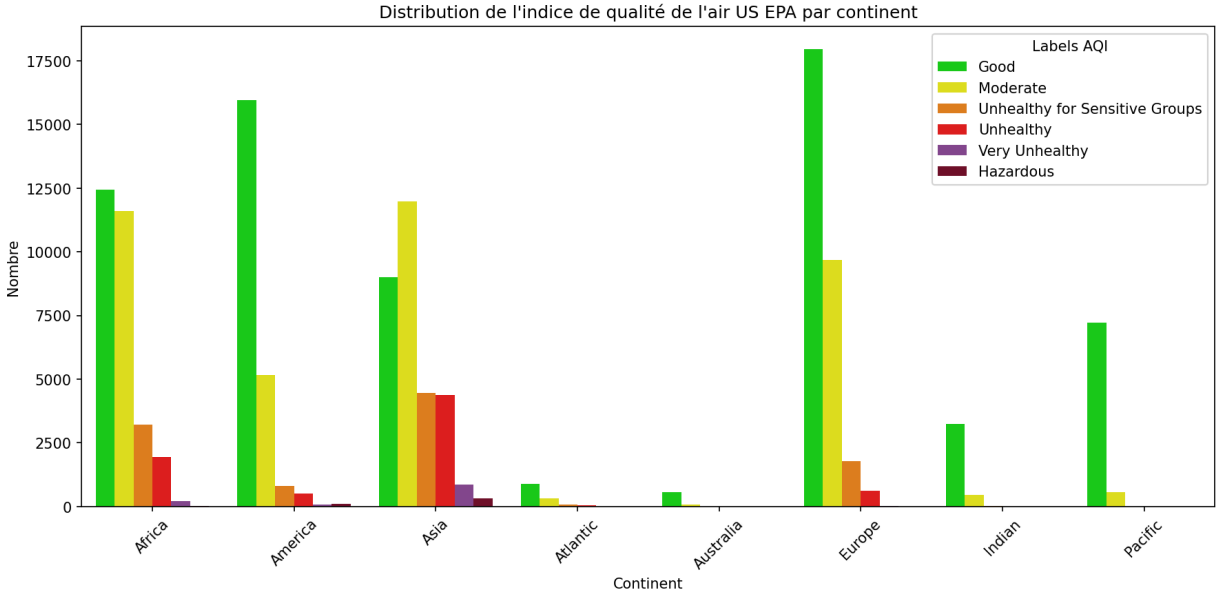
```python
df_2['year'] = df_2['date'].dt.year
df_2['month'] = df_2['date'].dt.month
df_2['day'] = df_2['date'].dt.day
df_2['hour'] = df_2['date'].dt.hour
```

3.4.2. Visualisations

Après le pré-traitement, il est crucial de visualiser le jeu de données final. Cette étape valide l'efficacité de notre nettoyage et permet d'observer l'apport des nouvelles variables temporelles et géographiques.

Les graphiques suivants illustrent les tendances sur lesquelles nos algorithmes vont désormais s'appuyer.

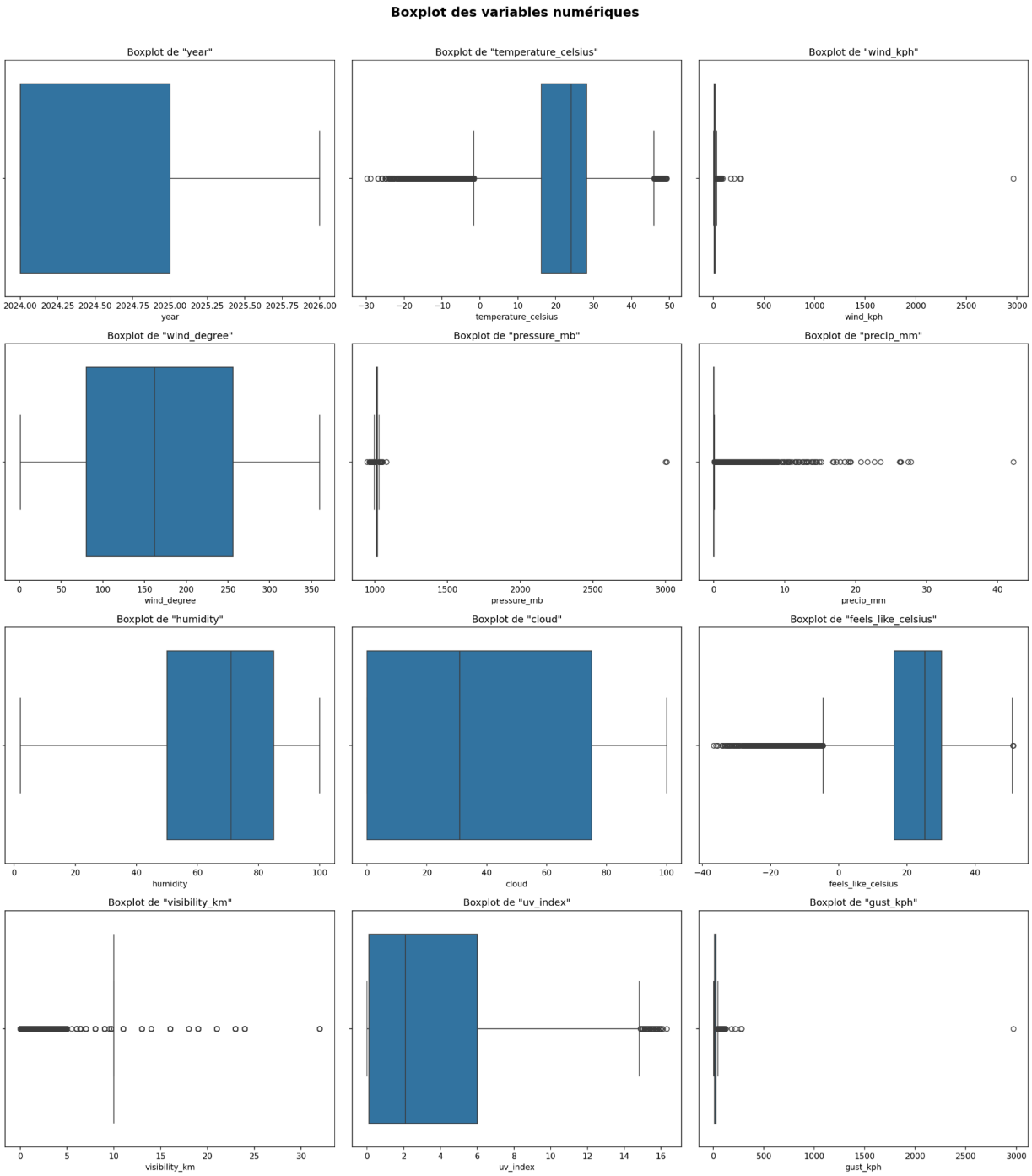
Distribution de l’indice de qualité de l’air US EPA par continent



→ **Distribution de l'AQI par continent :** Cette visualisation présente la qualité de l'air segmentée par zone géographique mondiale. Ce graphique a pu être généré uniquement parce que nous avons simplifié la variable complexe `timezone` pour ne conserver que le continent. On remarque que l'Asie présente les proportions les plus élevées de catégories critiques ('Unhealthy' à 'Hazardous'), ce qui est en totale cohérence avec notre précédente analyse du 'Top 10 des pays par AQI' où les puissances industrielles asiatiques dominaient le classement. À l'inverse, l'Europe et l'Amérique affichent une prédominance de la catégorie 'Good', indiquant une qualité de l'air globalement plus stable sur les périodes enregistrées.

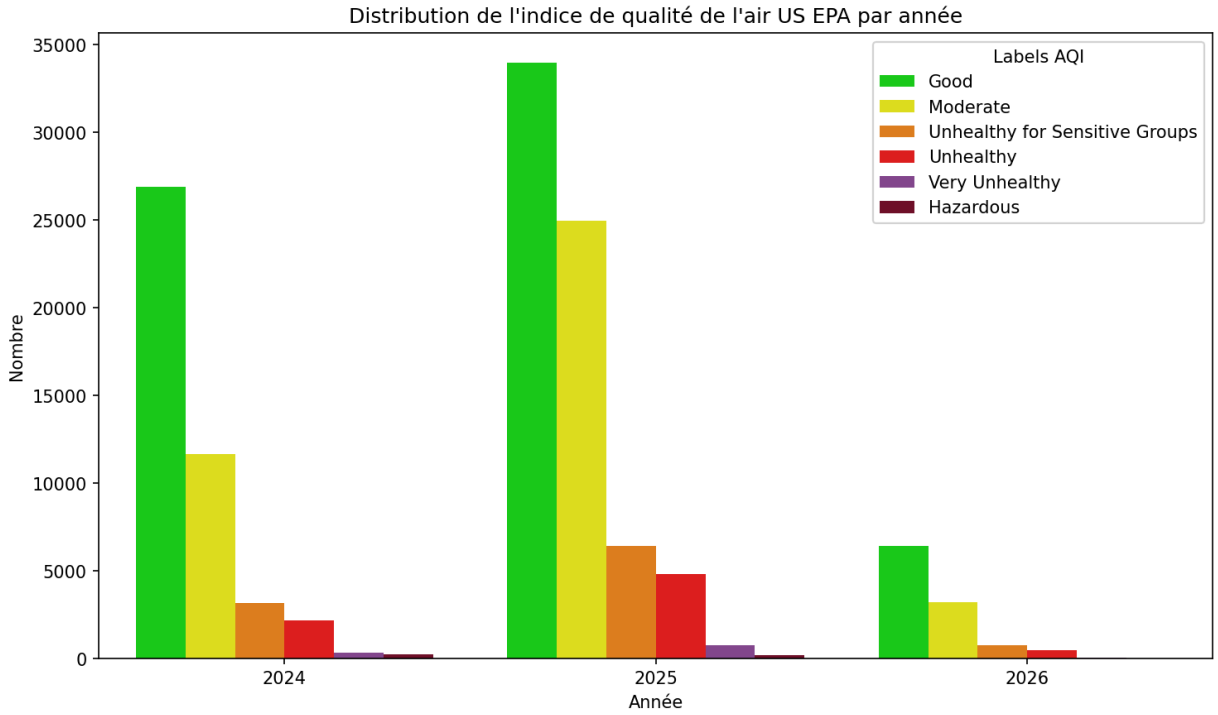
—

Boxplots des variables numériques



→ **Boxplots des variables numériques :** Ces graphiques permettent de visualiser la dispersion des données météo et d'identifier les valeurs atypiques (*outliers*). On remarque que des variables comme la vitesse du vent (`wind_kph`) ou les rafales (`gust_kph`) présentent de nombreux points extrêmes. Cette observation est tout à fait cohérente : les phénomènes météorologiques, tels que les tempêtes ou les rafales soudaines, génèrent naturellement des valeurs très éloignées de la moyenne. Ce nettoyage visuel confirme que nos variables physiques sont désormais bien typées et capturent fidèlement la réalité des contrastes climatiques mondiaux avant la modélisation.

Distribution de l'indice de qualité de l'air US EPA par année



→ **Distribution de l'AQI par année :** Ce graphique, rendu possible par l'extraction de la variable `year` lors du pré-traitement, permet de comparer la qualité de l'air sur trois années. On observe une forte cohérence entre 2024 et 2025, avec une prédominance de l'air sain ('Good'). En revanche, l'année 2026 affiche un volume de données nettement inférieur. Cela s'explique par le fait que 2026 est l'année en cours, les relevés ne couvrant donc qu'une période partielle au moment de la constitution du dataset. Cette visualisation valide la stabilité globale de nos données temporelles avant de passer à la modélisation.

Avec un jeu de données désormais propre, enrichi et validé visuellement, nous disposons d'une base solide pour la modélisation. L'enjeu est maintenant de transformer ces données en un modèle prédictif capable de classer la qualité de l'air avec précision.

3.5. Modélisation

Cette section constitue le cœur de notre étude. Notre objectif est de tester la capacité de différents algorithmes à prédire les catégories de qualité de l'air en nous appuyant sur l'utilisation exclusive des variables météorologiques.

La démarche se décompose comme suit :

- Division du dataset : Nous séparons nos données en un ensemble d'entraînement (train) pour permettre aux modèles d'apprendre, et un ensemble de test (test) pour évaluer leurs performances sur des données jamais vues.
- Comparaison d'algorithmes : Nous avons sélectionné trois modèles (dont nous avons parlé précédemment) aux propriétés complémentaires :
    - **Régression Logistique**
    - **Random Forest**
    - **XGBoost**

En comparant ces modèles, nous pourrons déterminer lequel est le plus apte à répondre à notre problématique et valider si les variables météorologiques suffisent à anticiper le niveau de pollution.

3.5.1. Split

Nous avons effectué un split des données.

Le Split (ou découpage) est une procédure fondamentale en Machine Learning qui consiste à diviser le jeu de données initial en deux sous-ensembles distincts et indépendants :

- **Le jeu d'entraînement (*Train set*) :** Constitué de 80% de nos données, c'est la base sur laquelle l'algorithme 's'exerce' à trouver des corrélations entre les variables météo/polluants et l'indice de qualité de l'air.
- **Le jeu de test (*Test set*) :** Regroupant les 20% restants, ces données sont strictement isolées durant toute la phase d'apprentissage. Elles servent de test final pour évaluer la capacité du modèle à prédire correctement sur des situations réelles qu'il n'a encore jamais rencontrées.

Cette séparation est indispensable pour détecter l'overfitting. En utilisant la fonction `train_test_split` de la bibliothèque `Scikit-Learn`, nous garantissons que ce découpage est effectué de manière aléatoire et reproductible.

```python
target = 'air_quality_us-epa-index'

# Définir les listes de features
meteo_features = [
    'temperature_celsius',
    'wind_kph',
    'wind_degree',
    'pressure_mb',
    'precip_mm',
    'humidity',
    'cloud',
    'feels_like_celsius',
    'visibility_km',
    'uv_index',
    'gust_kph',
]

pollutant_features = [
    'air_quality_Carbon_Monoxide',
    'air_quality_Ozone',
    'air_quality_Nitrogen_dioxide',
```

```python
    'air_quality_Sulphur_dioxide',
    'air_quality_PM2.5',
    'air_quality_PM10',
]

all_features = meteo_features + pollutant_features

# Split une seule fois (sur les index)
train_df, test_df = train_test_split(
    df_2,
    test_size=0.2,
    random_state=42,
    stratify=df_2[target]
)

# y
y_train = train_df[target]
y_test  = test_df[target]

# X
X_train_meteo = train_df[meteo_features]
X_test_meteo  = test_df[meteo_features]
```

3.5.2. Modèles

Nous avons entraîné trois modèles.

Pour évaluer la performance de nos trois modèles, nous avons utilisé un ensemble de métriques de classification complémentaires pour obtenir une vision précise de leur efficacité.

Logistic Regression

```python
logreg_params = {
    'max_iter': 3000,
    'C': 1.0,
    'solver': 'lbfgs',
    'class_weight': 'balanced'
}
```

	model	accuracy	precision_class1	recall_class1	f1_class1	roc_auc_ovr
0	LogReg_Model_B_Meteo_only	0.394647	0.699619	0.559685	0.621877	0.686845

Random forest

```python
# Paramètres RF
rf_params = {
    'n_estimators': 300,
    'max_depth': None,
    'min_samples_split': 2,
    'min_samples_leaf': 1,
    'n_jobs': -1,
    'class_weight': 'balanced'
}
```

	model	accuracy	precision_class1	recall_class1	f1_class1	roc_auc_ovr
0	RF_Model_B_Meteo_only	0.655795	0.719068	0.876468	0.790004	0.862819

XGBoost

```python
# Paramètres XGBoost
xgb_params = {
    'n_estimators': 500,
    'max_depth': 6,
    'learning_rate': 0.05,
    'subsample': 0.9,
    'colsample_bytree': 0.9,
    'reg_lambda': 1.0
}
```

	model	accuracy	precision_class1	recall_class1	f1_class1	roc_auc_ovr
0	XGB_Meteo_only	0.638639	0.716506	0.856623	0.780324	0.857773

—

Analyse et choix du modèle final

Les métriques utilisées sont :

- Accuracy : Proportion totale de prédictions correctes sur l'ensemble des classes.
- Precision : Capacité du modèle à ne pas classer comme 'bon' un air qui serait en réalité pollué (évite les faux positifs).
- Recall / Sensibilité : Capacité du modèle à détecter le maximum d'épisodes où l'air est réellement sain (évite les faux négatifs).

- F1-Score : Équilibre entre précision et rappel, particulièrement utile ici pour juger de la performance sur la classe majoritaire 'Good'.
- ROC-AUC : Performance globale du modèle à distinguer chaque classe par rapport aux autres. Plus le score est proche de 1, plus le pouvoir séparateur du modèle est fort.

Le Random Forest l'emporte avec une Accuracy de 65,6% et un ROC-AUC de 0,86. Sa force réside dans son Recall élevé (0,87) pour la Classe 1, garantissant que le modèle identifie très efficacement les zones de confort respiratoire. Contrairement à la Régression Logistique, il parvient à capter les interactions physiques complexes entre les variables pour en déduire un état de pollution probable.

C'est donc ce modèle que nous retenons pour notre solution finale !

3.5.3. Cross-validation

Afin de garantir la fiabilité de notre modèle et de s'assurer de sa capacité de généralisation, nous utilisons une Cross-Validation (K-Fold). Plutôt que de se fier à un seul découpage des données, cette méthode divise le jeu de données en k parties (ici 5). Le modèle est entraîné k fois, en utilisant à chaque fois une partie différente comme jeu de test et les autres comme jeu d'entraînement.

Cette approche permet de :

- Réduire le biais lié au découpage aléatoire des données.
- Vérifier la stabilité des performances (en observant l'écart-type des scores).
- Donner une estimation plus robuste de la performance réelle du modèle sur des données futures.

```python
# 4. Cross-validation stratifiée
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
cv_results = cross_validate(
    rf, X, y,
    scoring=scoring,
    cv=cv,
    n_jobs=-1,
    return_train_score=False
)
```

Le résultat de notre Cross-validation est le suivant :

```python
=== RandomForest 5-fold CV (stratified) ===
                          mean     std
test_accuracy           0.6565  0.0011
test_precision_class1   0.7167  0.0011
```

```
test_recall_class1     0.8767  0.0047
test_f1_class1         0.7887  0.0022
test_roc_auc_ovr          NaN     NaN

Scores par fold :
   fold  accuracy  precision_class1  recall_class1  f1_class1  roc_auc_ovr
0     1    0.6563            0.7169         0.8800     0.7901          NaN
1     2    0.6551            0.7175         0.8722     0.7873          NaN
2     3    0.6579            0.7171         0.8831     0.7915          NaN
3     4    0.6570            0.7170         0.8756     0.7884          NaN
4     5    0.6563            0.7148         0.8728     0.7859          NaN
```

—

Justification des ‘`NaN`’ pour la métrique ‘roc_auc_ovr’ en validation croisée

Les valeurs ‘`NaN`’ observées pour la métrique *‘roc_auc_ovr’* lors de la validation croisée stratifiée s'expliquent par la distribution déséquilibrée des classes dans le jeu de données. En validation croisée à 5 folds, chaque fold de test représente 20 % du dataset. Or, même avec une stratification, certaines classes minoritaires peuvent ne contenir que 0 ou 1 observation dans un fold de test donné. Dans ce cas, roc_auc_score ne peut mathématiquement pas calculer l'aire sous la courbe ROC pour cette classe, ce qui produit une valeur indéfinie (`NaN`).

—

La validation croisée stratifiée en 5 folds montre des performances très stables du modèle Random Forest.

L’accuracy moyenne obtenue est de 0.6565 avec un écart-type très faible (± 0.0011), ce qui indique une grande stabilité des performances d’un fold à l’autre.

De même, les métriques ciblées sur la classe 1 (precision, recall et F1-score) présentent des variations très limitées entre les différents splits, confirmant la robustesse du modèle face aux différentes partitions des données.

Cette faible variance entre les folds suggère que le modèle généralise de manière cohérente et qu’il ne dépend pas fortement d’un découpage particulier du jeu de données.

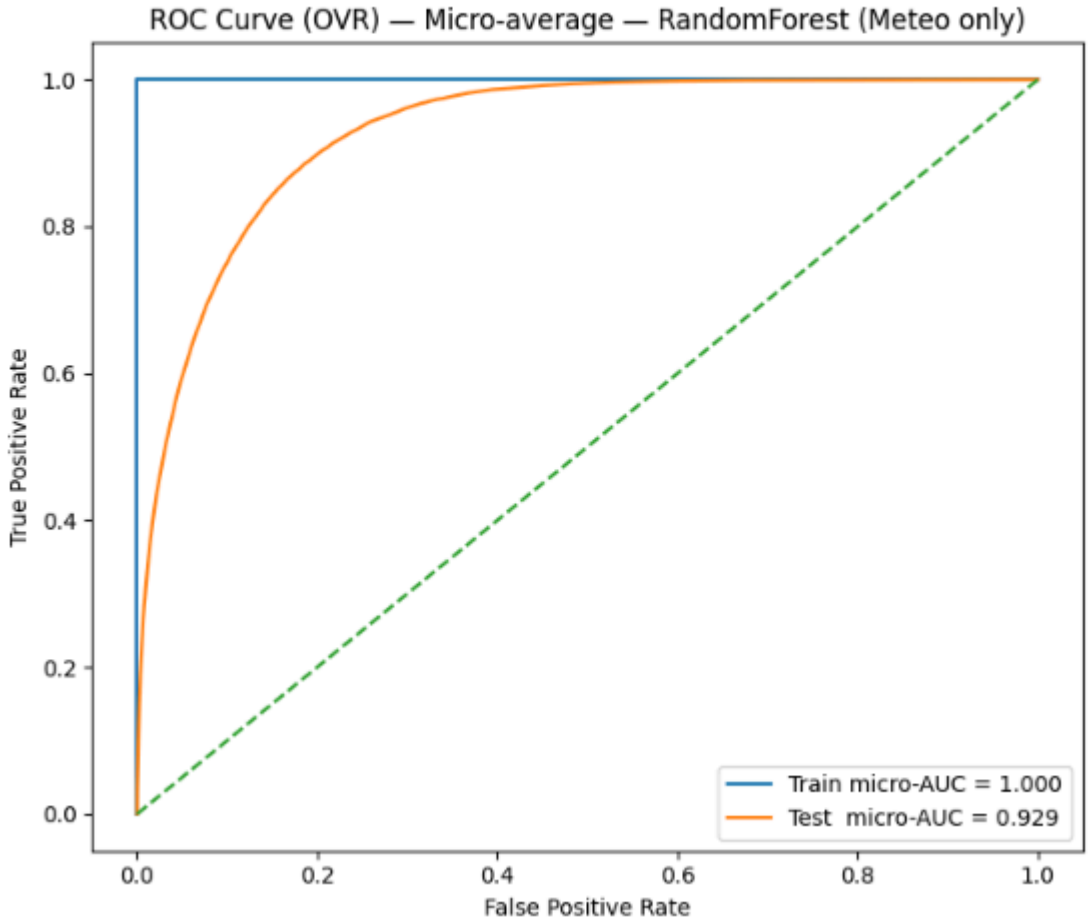
3.5.3. Overfitting

L'overfitting survient lorsqu'un modèle apprend 'par cœur' les données d'entraînement, y compris leur bruit et leurs particularités aléatoires, au lieu de saisir la logique générale. Un tel modèle affichera des performances quasi parfaites sur le jeu d'entraînement mais échouera à prédire correctement les données du jeu de test.

Pour notre modèle de Random Forest, il est essentiel de surveiller l'écart entre les scores de ces deux jeux : une précision très élevée en entraînement couplée à une chute significative sur le test confirmerait un surapprentissage, rendant le modèle inutilisable en conditions réelles.

Nous avons généré le graphique suivant :

```
=== AUC Train vs Test (OVR) — Meteo only ===
Micro-AUC    : Train=1.000 | Test=0.929
Macro-AUC    : Train=1.000 | Test=0.863
Weighted-AUC : Train=1.000 | Test=0.820
```



Le modèle Random Forest atteint une performance parfaite sur l'ensemble d'entraînement (AUC = 1.000), ce qui est un comportement classique des modèles à base d'arbres non contraints, capables de mémoriser précisément les observations du jeu d'apprentissage.

Toutefois, l'évaluation sur le jeu de test montre une AUC micro de 0.929, une AUC macro de 0.863 et une AUC pondérée de 0.820.

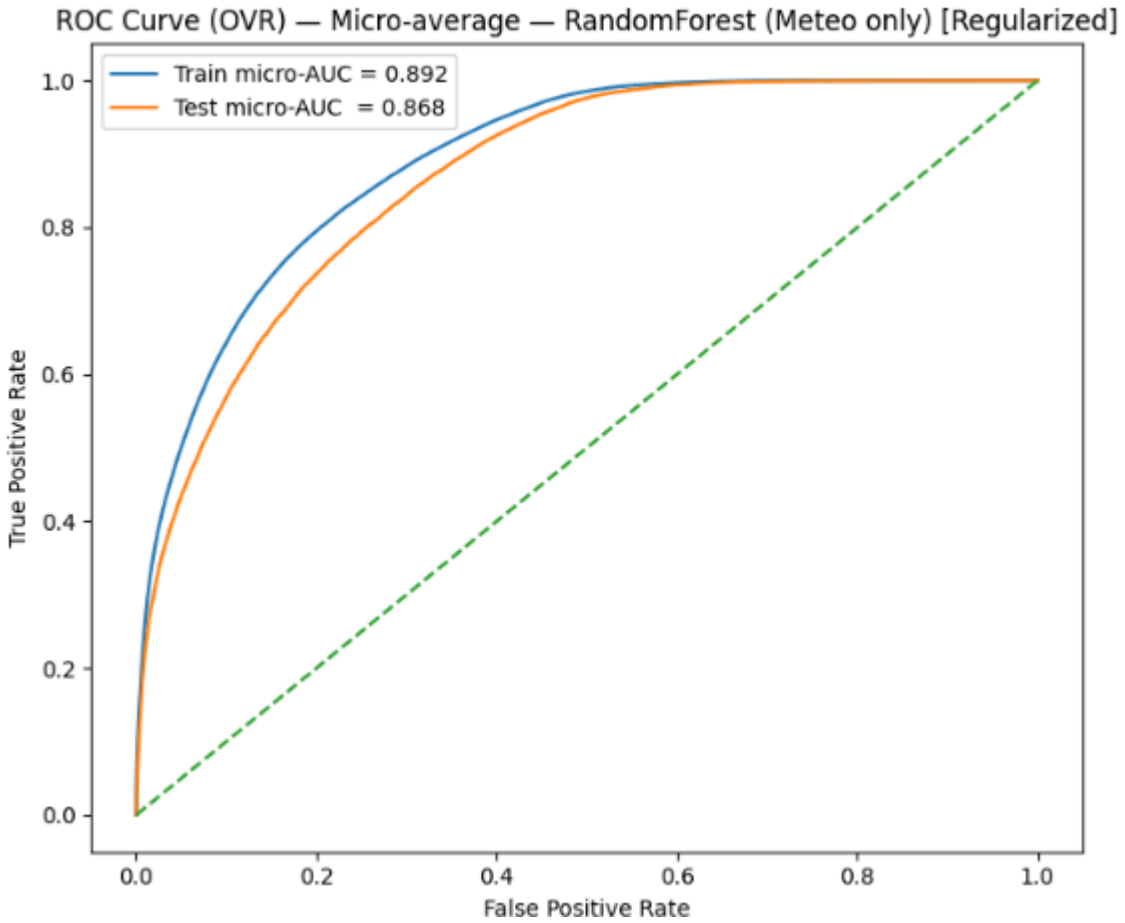
Bien qu'un écart existe entre les performances train et test, les résultats obtenus sur les données non vues restent élevés, indiquant une bonne capacité de généralisation du modèle.

L'overfitting observé est donc modéré et n'altère pas significativement la performance prédictive du modèle.

Nous avons quand même tenté de réduire cet overfitting.

Nous avons alors obtenu ceci :

```
OOB score (approx generalisation): 0.528
```



Afin de limiter la capacité de mémorisation excessive du modèle Random Forest initial, plusieurs contraintes ont été introduites (limitation de la profondeur des arbres, augmentation du nombre minimum d'échantillons par feuille et par split).

Le modèle régularisé obtient :

- AUC train = 0.892
- AUC test = 0.868

L'écart entre train et test est désormais réduit (≈ 0.024 contre ≈ 0.071 précédemment), ce qui indique une meilleure capacité de généralisation.

Choix final du modèle

Deux versions du modèle Random Forest ont été évaluées : une version non contrainte présentant un certain niveau d'overfitting, et une version régularisée visant à réduire cet écart.

Bien que le modèle régularisé présente un écart train/test plus faible, il entraîne une diminution significative des performances prédictives (baisse de l'AUC et de l'accuracy sur le jeu de test).

Le modèle initial, bien que montrant une capacité de mémorisation élevée sur le jeu d'entraînement, conserve d'excellentes performances sur le jeu de test (AUC ≈ 0.92) et une stabilité satisfaisante en validation croisée.

L’overfitting observé est donc considéré comme modéré et acceptable au regard du gain substantiel en performance prédictive.

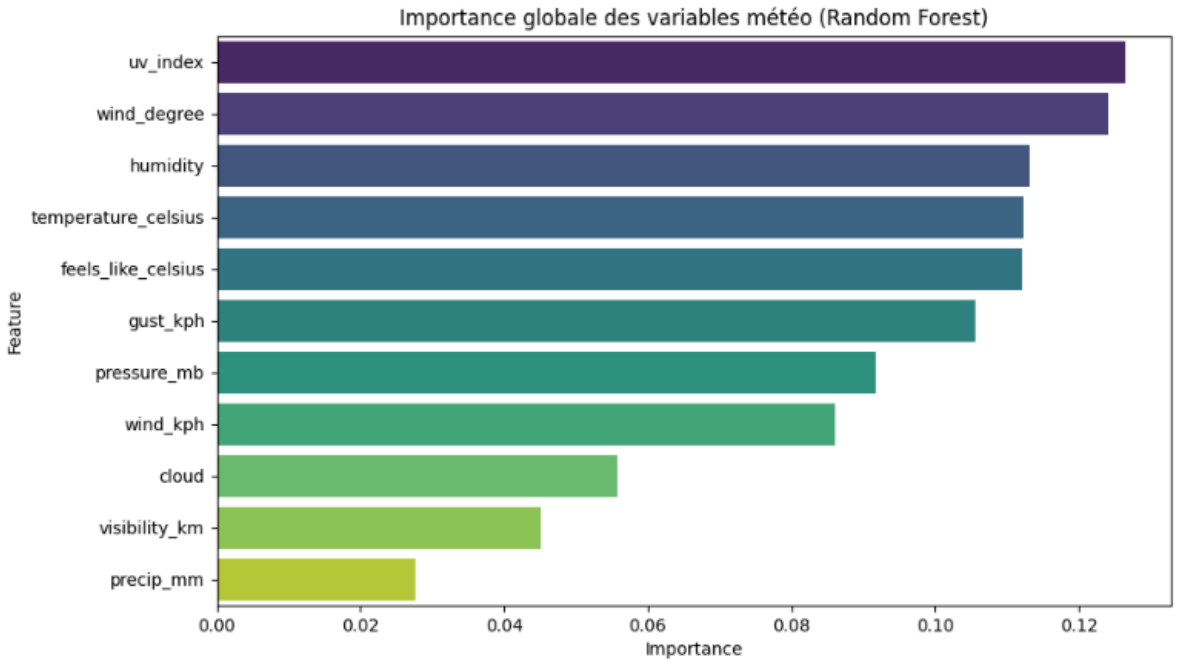
Le modèle Random Forest non contraint est ainsi retenu comme modèle final pour l’analyse météorologique.

Le modèle retenu est le suivant :

```python
best_rf_params = {
    'n_estimators': 300,
    'max_depth': None,
    'min_samples_split': 2,
    'min_samples_leaf': 1,
    'n_jobs': -1,
    'class_weight': 'balanced',
    'random_state': 42
}
final_model = RandomForestClassifier(**best_rf_params)
final_model.fit(X_train_meteo, y_train.astype(int))
```

3.5.4. Interprétabilité

Pour comprendre comment notre Random Forest prend ses décisions, nous utilisons l'importance de Gini (Mean Decrease Impurity). Cette méthode calcule à quel point chaque variable météorologique contribue à la pureté des nœuds de l'arbre lors de la classification. Plus une variable a un score élevé, plus elle est déterminante pour prédire précisément la catégorie de qualité de l'air.



Analyse de l'importance des variables (Feature Importance)

Ce graphique présente l'importance relative de chaque variable météorologique dans le processus de décision du modèle Random Forest pour prédire la catégorie de qualité de l'air .

Les observations principales sont :

Prédominance des facteurs de rayonnement et de circulation :

- L'indice UV (`uv_index`) apparaît comme la variable la plus influente (env. 12,5 %). Cela s'explique physiquement par le fait que le rayonnement solaire favorise les réactions chimiques dans l'atmosphère, notamment la formation d'ozone troposphérique (un polluant clé).
- La direction du vent (`wind_degree`) suit de près. Elle est cruciale car elle détermine la provenance des masses d'air (zones industrielles ou zones naturelles) et donc le transport des polluants vers une localité donnée.

Impact des conditions thermodynamiques :

- L'objectif de cette étude était de déterminer dans quelle mesure les variables météorologiques permettent d'anticiper la catégorie globale de qualité de l'air, en faisant abstraction des mesures directes de polluants.

Variables de dispersion mécanique :

- La vitesse des rafales (`gust_kph`) et la pression atmosphérique (`pressure_mb`) montrent une importance significative. Une pression élevée (anticyclone) est souvent synonyme de stabilité de l'air et donc d'accumulation de pollution au sol.

Variables moins contributives :

- La nébulosité (`cloud`), la visibilité (`visibility_km`) et les précipitations (`precip_mm`) ferment le classement. Bien que les pluies aient un effet de 'lessivage' de l'air, elles semblent ici moins discriminantes que les facteurs de rayonnement et de vent pour prédire l'indice global sur l'ensemble du dataset.

3.5.5. Conclusion

L'objectif de cette étude était de déterminer dans quelle mesure les variables météorologiques permettent d'anticiper la catégorie globale de qualité de l'air, en faisant abstraction des mesures directes de polluants.

Au terme de cette analyse, plusieurs conclusions majeures s'imposent.

Performance et fiabilité du modèle

Pour répondre à cette problématique, nous avons privilégié un modèle de forêt aléatoire (Random Forest). Ce choix s'est avéré concluant puisque le modèle atteint une accuracy de 65,6 % sur le jeu de test. Plus significatif encore pour un problème multi-classes, l'AUC s'élève

à 0,86, ce qui démontre une excellente capacité du modèle à distinguer les différentes catégories de qualité de l'air uniquement à partir de données météo. Bien qu'une légère tendance à l'overfitting ait été notée, la stabilité des scores en validation croisée confirme la robustesse de ces résultats.

Les leviers météorologiques de la pollution

Grâce à un outil d'interprétabilité (Gini Importance), nous avons identifié les facteurs qui contribuent de plus à la qualité de l'air dans notre modèle :

- Le rayonnement solaire : C'est la variable la plus prédictive. Cela souligne le rôle crucial de l'énergie solaire dans la formation de polluants secondaires comme l'ozone.
- La dynamique des vents : La direction et la force du vent déterminent si les polluants sont dispersés ou, au contraire, transportés depuis des zones industrielles vers les centres urbains.
- Les conditions thermodynamiques (Humidité et Pression) : Ces variables influent sur la stabilité de la masse d'air. Une pression haute et une humidité spécifique favorisent souvent la stagnation des particules fines au niveau du sol.

Réponse à la problématique

Les résultats montrent que les variables météorologiques sont des indicateurs avancés performants de la qualité de l'air. Si elles ne remplacent pas les capteurs chimiques pour une mesure précise, elles permettent d'anticiper la catégorie globale (Good, Moderate, Unhealthy, etc.) avec une fiabilité élevée.
Cette capacité d'anticipation repose sur une réalité physique : la météo ne se contente pas d'accompagner la pollution, elle en est le moteur ou le frein. Les conditions atmosphériques agissent comme un réacteur chimique géant (via les UV) et un système de distribution (via le vent et la pression).
En conclusion, cette étude démontre qu'il est possible de construire des systèmes d'alerte précoce basés uniquement sur des prévisions météorologiques. Dans des régions où les stations de mesure de qualité de l'air sont rares ou coûteuses à entretenir, l'utilisation de modèles prédictifs basés sur la météo constitue une alternative crédible et efficace pour protéger la santé publique.

4. Solution Opérationnelle : De l'Algorithme à l'Application

Cette dernière étape marque la concrétisation de notre étude : la transformation d'un modèle statistique abstrait en un outil fonctionnel, interactif et accessible.

4.1. Architecture Logicielle et Environnement

Pour transformer notre modèle de Machine Learning en une interface web, nous avons utilisé l'écosystème Python et la bibliothèque `Streamlit`. L'architecture repose sur plusieurs piliers techniques :

- **Gestion du modèle :** La librairie `joblib` permet de charger le modèle Random Forest préalablement entraîné et sauvegardé au format `.pkl`.
- **Performance :** L'utilisation du décorateur `@st.cache_resource` est cruciale pour l'expérience utilisateur ; il maintient le modèle en mémoire vive, évitant un rechargement lourd à chaque interaction.
- **Données en temps réel :** Via la librairie `requests`, l'application communique avec l'API OpenWeatherMap pour récupérer les conditions météorologiques mondiales instantanées.
- **Sécurité :** Les clés d'API sensibles sont protégées via des variables d'environnement (`.env`), garantissant qu'aucune information critique n'est exposée dans le code source.

4.2. Interface Utilisateur : Les quatre piliers de l'outil

L'interface a été segmentée en quatres pages pour accompagner l'utilisateur de la compréhension théorique à l'utilisation pratique :

1. **Le Notebook :** Intègre le rendu HTML complet de la phase de recherche, permettant de consulter le code et les graphiques d'exploration sans quitter l'outil.
2. **Synthèse Décisionnelle :** Répond à la problématique en vulgarisant les résultats (Accuracy, ROC-AUC) pour les rendre accessibles aux non-spécialistes.
3. **Le Simulateur :** Permet de manipuler des curseurs (*sliders*) pour tester des scénarios hypothétiques (ex. : canicule sans vent) et observer instantanément l'impact sur l'AQI.
4. **Prédiction par ville :** L'outil opérationnel qui 'bouche les trous' des capteurs physiques en estimant la qualité de l'air de n'importe quelle localité à partir de sa météo en direct.

4.3. Accessibilité et Reproductibilité

Afin de garantir la transparence et la portabilité de la solution, l'intégralité du projet est structurée pour être déployée instantanément.

- **Hébergement du Code :** Le code source complet, incluant le fichier principal de l'interface (`app.py`), le modèle entraîné (`.pkl`) et les environnements virtuels nécessaires, est hébergé sur un dépôt GitHub public (https://github.com/leagravellard/Memoire).
- **Lancement de l'application :** Pour un usage local, l'utilisateur peut cloner le répertoire et lancer l'interface via une commande simplifiée ‘`streamlit run`

`src/app.py'`, après avoir installé les dépendances listées dans le fichier '`requirements.txt'`.

5. Conclusion et Perspectives

Au terme de cette étude, nous sommes en mesure de répondre à notre problématique centrale : "**Modélisation prédictive de la qualité de l'air : dans quelle mesure le Machine Learning permet-il de prédire la qualité de l'air à partir des seules données météorologiques, indépendamment des mesures directes de polluants ?"**.

Nos travaux ont démontré que la météo ne se contente pas d'accompagner la pollution, elle en est le véritable moteur physique et chimique. En utilisant un modèle de **Random Forest**, nous avons réussi à prédire l'indice de qualité de l'air avec une **précision de 65,6 %** et un **score ROC-AUC de 0,86** en nous appuyant exclusivement sur des données météorologiques.

L'analyse de l'importance des variables a révélé que le **rayonnement UV** et la **dynamique des vents** sont les facteurs les plus déterminants. Bien que ces 'capteurs virtuels' ne remplacent pas la précision des mesures chimiques, ils offrent une alternative fiable, peu coûteuse et immédiatement déployable pour les régions souffrant d'un manque d'infrastructures de surveillance.

Au-delà de la validation statistique, ce mémoire a abouti à la création d'une application interactive sous '`Streamlit`'. Cet outil matérialise la transition entre la théorie algorithmique et l'usage citoyen :

- **Accessibilité :** Grâce au simulateur, n'importe quel utilisateur peut comprendre l'impact d'une hausse de température ou d'une absence de vent sur l'air qu'il respire.
- **Perspectives d'industrialisation :** Bien que l'outil fonctionne actuellement dans un environnement local, sa conception logicielle a été pensée pour l'évolution. La structure modulaire du code facilite une future conteneurisation via Docker, ouvrant la voie à un déploiement simplifié sur des solutions Cloud (ex. : AWS).

Si les résultats sont encourageants, plusieurs pistes d'amélioration pourraient être explorées pour accroître la performance du système :

1. **Intégration de la topographie :** Le modèle actuel ne prend pas en compte le relief ou la densité des bâtiments (canyons urbains). L'ajout de variables géospatiales permettrait d'affiner les prédictions au niveau local.
2. **Saisonnalité et historique :** L'utilisation de réseaux de neurones récurrents (de type LSTM[13]) pourrait permettre de mieux capturer les dépendances temporelles et les effets d'accumulation de la pollution sur plusieurs jours.

[13] **Réseaux de neurones récurrents (LSTM) :** Le *Long Short-Term Memory* est une architecture de Deep Learning spécialisée dans le traitement de séquences temporelles. Contrairement aux modèles classiques, le LSTM possède une « mémoire » capable de retenir des informations sur de longues périodes. Dans notre contexte, il permettrait au modèle de comprendre que la pollution d'aujourd'hui dépend non seulement de la météo actuelle, mais aussi de l'accumulation des polluants observée les jours précédents.

3. **Données de trafic :** Comme l'illustre le benchmark de Google Air Quality API, la fusion de nos données météorologiques avec des flux de trafic routier en temps réel permettrait de 'boucher les trous' restants de notre modèle.

En conclusion, ce projet prouve que l'alliance entre l'Open Data météorologique et le Machine Learning constitue un levier puissant pour la santé publique. En démocratisant l'accès à l'information environnementale, nous offrons aux populations vulnérables un outil de vigilance face aux défis climatiques et urbains de demain.

6. Bibliographie et Webographie

6.1. Institutions, Rapports et Données

- **Airparif.** (s. d.). *Surveillance de la qualité de l'air en Île-de-France*. https://www.airparif.fr/
- **Copernicus Atmosphere Monitoring Service (CAMS).** (s. d.). *Global atmospheric composition monitoring*. https://atmosphere.copernicus.eu/
- **INERIS.** (s. d.). *Modélisation et prévision de la qualité de l'air ambiant*. https://www.ineris.fr/fr/risques/dossiers-thematiques/qualite-air/qualite-air-ambiant/modelisationprevision-qualite-air
- **Kaggle.** (2023). *Global Weather Repository Dataset*. https://www.kaggle.com/datasets/nelgiriyewithana/global-weather-repository
- **Nations Unies (ONU).** (2025). *World Urbanization Prospects 2025: Summary of Results*. chrome-extension://efaidnbmnnnibpcajpcglclefindmkaj/https://www.un.org/development/desa/pd/sites/www.un.org.development.desa.pd/files/undesa_pd_2025_wup2025_summary_of_results.pdf
- **Organisation Mondiale de la Santé (OMS).** (2014). *7 millions de décès prématurés sont liés chaque année à la pollution de l'air*. https://www.who.int/fr/news/item/25-03-2014-7-million-premature-deaths-annually-linked-to-air-pollution
- **Programme des Nations Unies pour l'environnement (PNUE).** (s. d.). *Comment mesure-t-on la qualité de l'air ?* https://www.unep.org/fr/actualites-et-recits/recit/comment-mesure-t-la-qualite-de-lair

6.2. Littérature Scientifique et Théorie du Machine Learning

- **Breiman, L., Friedman, J., Olshen, R., & Stone, C.** (1984). *Classification and Regression Trees*. Taylor & Francis. https://www.taylorfrancis.com/books/mono/10.1201/9781315139470/
- **Chen, T., & Guestrin, C.** (2016). *XGBoost: A Scalable Tree Boosting System*. Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. https://arxiv.org/abs/1603.02754
- **Lary, D. J., et al.** (2015). *Using Machine Learning to Estimate Global PM2.5 for Environmental Health Studies*. *Environmental Health Perspectives*.
- **Parlement Européen.** (2020). *Intelligence artificielle : définition et utilisation*. https://www.europarl.europa.eu/topics/fr/article/20200827STO85804/intelligence-artificielle-definition-et-utilisation
- **Wikipedia.** (s. d.). *Intelligence artificielle*. https://fr.wikipedia.org/wiki/Intelligence_artificielle

6.3. Documentation Technique et Bibliothèques Python

- **IBM.** (s. d.). *Guides pédagogiques : Régression logistique, Arbres de décision et Random Forest*. https://www.ibm.com/fr-fr/think/topics/
- **Joblib Development Team.** (s. d.). *Joblib: running Python functions as pipeline jobs*. https://joblib.readthedocs.io/
- **Pandas Development Team.** (s. d.). *Pandas Documentation*. https://pandas.pydata.org/docs/
- **Python Software Foundation.** (s. d.). *Applications for Python*. https://www.python.org/about/apps/
- **Scikit-Learn.** (s. d.). *User Guide : Ensemble Methods & Linear Models*. https://scikit-learn.org/stable/index.html
- **Seaborn.** (s. d.). *Statistical data visualization*. https://seaborn.pydata.org/
- **Streamlit.** (s. d.). *Streamlit Documentation*. https://docs.streamlit.io/

5.4. Concepts Généraux et Webographie

- **CNRTL.** (s. d.). *Définition : Industrialisation*. https://www.cnrtl.fr/definition/industrialisation
- **Géoconfluences (ENS Lyon).** (s. d.). *Glossaire : Exode rural*. https://geoconfluences.ens-lyon.fr/glossaire/exode-rural-migrations-rurales-exode-urbain
- **IQAir.** (s. d.). What is AQI? Air Quality Index Scale Explained. https://www.iqair.com/ca-fr/newsroom/what-is-aqi
- **Météocentre.** (s. d.). *L'inversion thermique et son impact sur la pollution*. https://meteocentre.com/intermet/temperature/inversion.htm
- **MSD Manuals.** (s. d.). *Tableau de l'indice de qualité de l'air (IQA)*. https://www.msdmanuals.com/fr/accueil/multimedia/table/indice-de-qualit%C3%A9-de-l air-iqa
- **Wikipedia.** (s. d.). *Garbage in, garbage out (GIGO)*. https://fr.wikipedia.org/wiki/GIGO
- **Wikipedia.** (s. d.). *Urbanisation*. https://fr.wikipedia.org/wiki/Urbanisation